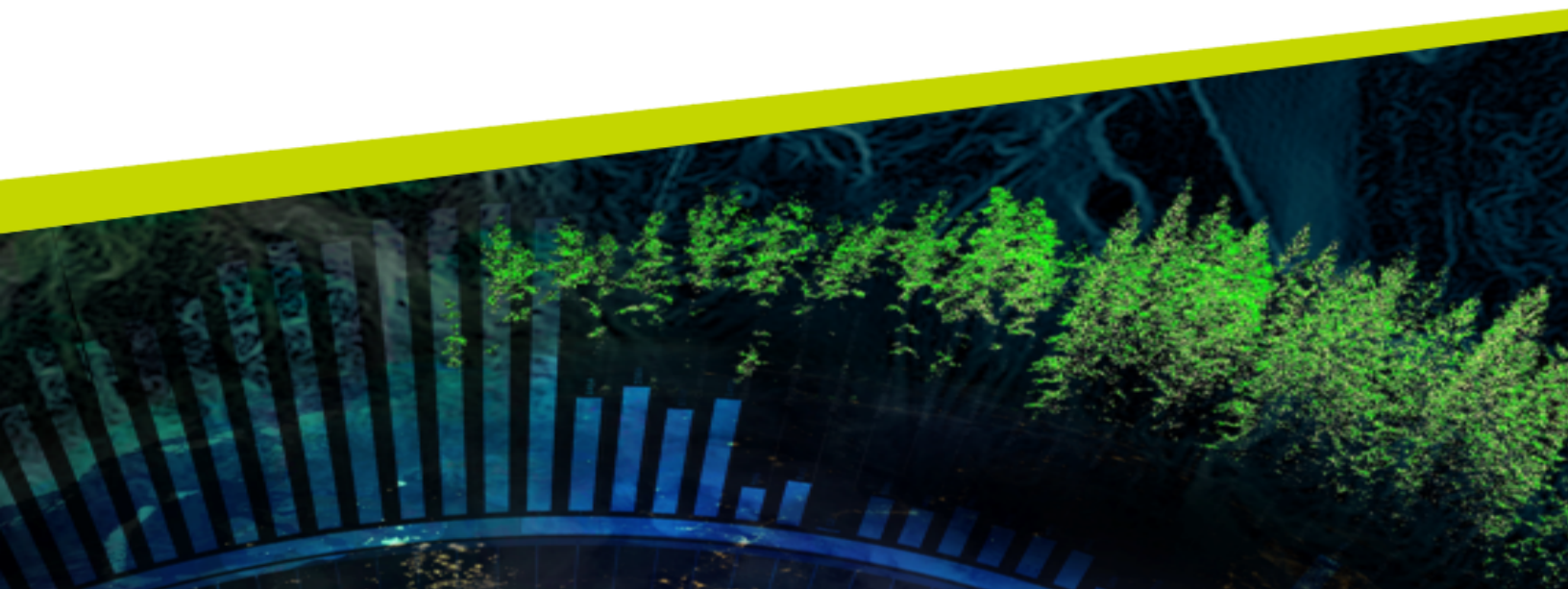




INFERENCIA Y MODELOS ESTADÍSTICOS

José Luis Jara V. y Jacqueline Köhler C.



CAPÍTULO 12. ALTERNATIVAS MÁS RECIENTES PARA ANALIZAR DATOS PROBLEMÁTICOS

Como ya sabemos, muchos procedimientos estadísticos requieren que los datos cumplan con ciertas propiedades o condiciones, lo que no siempre ocurre. En capítulos anteriores hemos visto que, ante este escenario, podemos intentar analizar datos transformados (sección 11.1) o usar como alternativa algún método no paramétrico (capítulo 8 y secciones 11.2 y 11.3). Pero estas no son las únicas opciones como veremos en este capítulo, donde abordaremos otras estrategias, que podemos usar cuando necesitamos analizar datos problemáticos, que son más recientes y que requieren cierto poder de cómputo.

12.1 REMUESTREO

Los **métodos basados en remuestreo** son una buena alternativa a emplear cuando necesitamos inferir sobre parámetros distintos a la media o la proporción, o bien cuando no se cumplen los supuestos sobre la distribución de los datos (como normalidad u homocedasticidad) o el conocimiento de parámetros poblacionales (como la varianza) que hacen las pruebas paramétricas estudiadas. También pueden ser útiles cuando las muestras son reducidas e insuficientes para asegurar que el Teorema del Límite Central está operando, lo que hace que las aproximaciones asintóticas en que se basan las pruebas clásicas puedan ser imprecisas.

La idea básica detrás del remuestreo es extraer **repetidamente** muestras desde un conjunto original de datos observados para obtener **información sobre la población** de la que provienen. A estas muestras de la muestra original se les conoce como **remuestras**.

En vez de depender de supuestos teóricos sobre la distribución de la población, el remuestreo utiliza las remuestras para **simular múltiples experimentos** y obtener un conjunto de estadísticas de ellos, desde donde se pueden obtener estimaciones de la variabilidad de un estadístico muestral, construir intervalos de confianza o docimar hipótesis.

Sin embargo, aunque el remuestreo relaja muchas condiciones paramétricas, no está libre de supuestos:

- La muestra original es **representativa de la población**, que es fundamental para que las estadísticas obtenidas con las remuestras sean válidas. Si la muestra original está sesgada, las remuestras también lo estarán, y las conclusiones que podamos obtener podrían ser equivocadas para la población en estudio.
- Las observaciones dentro de la muestra original son **independientes**. Si bien existen algunos métodos de remuestreo para datos con dependencias (como series de tiempo con autocorrelación o datos agrupados), las técnicas básicas, como las estudiadas en este capítulo, asumen la independencia entre los casos incluidos en la muestra.

La validez de las conclusiones obtenidas con la mayoría de las técnicas de remuestreo depende críticamente de que estos supuestos se cumplan razonablemente. También es necesario tener en cuenta que cada técnica de remuestreo puede hacer suposiciones propias, y que deben cumplirse para aplicarse con validez.

Pese a estas ventajas, los métodos basados en remuestreo realizan enormes cantidades de cálculos, por lo que en la práctica requieren de herramientas de software para su aplicación. Por otro lado, aunque existen métodos de remuestreo paramétricos y semiparamétricos, en este capítulo abordaremos las principales técnicas de **remuestreo no paramétrico**, basándonos en las ideas descritas por Amat Rodrigo (2016) y Hesterberg et al. (2003).

12.1.1 Bootstrapping

A partir de lo que hemos aprendido hasta ahora, deberíamos tener claro que en estadística el ideal es contar con varias muestras grandes. Pero muchas veces solo disponemos de una muestra relativamente pequeña.

Sin embargo, si esta muestra es representativa de la población, esperaríamos que las observaciones que ella contiene aparecieran con **frecuencias similares a las de la población**. El método de remuestreo **bootstrapping** se construye en torno a esta idea.

En general, si queremos inferir el valor de un parámetro de la población θ , hasta ahora lo hemos hecho a partir de un estimador puntual $\hat{\theta}$ calculado desde una muestra. Aplicar bootstrapping en este proceso de inferencia, en términos generales, sigue los siguientes pasos:

1. Crear una gran cantidad R de remuestras (cientos, miles, decenas de miles y hasta cientos de miles) a partir de la muestra original. Cada remuestra debe tener el **mismo tamaño** que la original y se construye mediante **muestreo con reposición**. Esto quiere decir que al seleccionar un caso desde la muestra original, este se devuelve a ella antes de tomar el siguiente, por lo que el caso podría ser reelegido.
2. Calcular el estadístico de interés $\hat{\theta}^*$ para cada una de las remuestras; aquí se usa “*” para indicar que corresponde a un **estadístico bootstrap**, es decir, obtenido desde una remuestra generada con bootstrapping. Estos estadísticos bootstrap producen una distribución empírica del estadístico $\hat{\theta}$, la que se conoce como **distribución bootstrap**.
3. Usar la distribución bootstrap para obtener información útil acerca de la forma, el centro y la variabilidad de la distribución muestral del estadístico de interés $\hat{\theta}$.

De esta forma, podemos obtener estadísticos bootstrap desde la distribución bootstrap. Por ejemplo, podemos obtener su media y error estándar por medio de las ecuaciones 12.1 y 12.2, respectivamente.

$$\bar{x}_{(\hat{\theta}^*, R)} = \frac{1}{R} \sum_{i=1}^R \hat{\theta}_i^* \quad (12.1)$$

$$SE_{(\hat{\theta}^*, R)} = \sqrt{\frac{\sum_{i=1}^R (\hat{\theta}_i^* - \bar{x}_{(\hat{\theta}^*, R)})^2}{R - 1}} \quad (12.2)$$

También es posible obtener fácilmente un intervalo de confianza para $\hat{\theta}$, simplemente se considera el rango de valores en torno al centro de la distribución muestral que cumple con el $100(1 - \alpha)\%$ confianza deseado.

Es importante remarcar que la distribución bootstrap se centra en el **estadístico observado** $\hat{\theta}$, y no, como desearíamos, en el parámetro θ . Este resultado teórico origina otro estadístico útil, llamado **sesgo**, *bias* en inglés, que corresponde al desplazamiento del estadístico $\hat{\theta}$ de la media de la distribución bootstrap que genera, y que está dado por la ecuación 12.3.

$$\delta_{(\hat{\theta}^*, R)} = \bar{x}_{(\hat{\theta}^*, R)} - \hat{\theta} \quad (12.3)$$

El mayor uso de bootstrapping apunta a construir intervalos de confianza más precisos para el parámetro θ a partir de la distribución bootstrap de $\hat{\theta}$, y no de la estimación puntual (un solo valor) que $\hat{\theta}$ entrega. A partir de allí, la técnica nos permite contrastar hipótesis.

Si bien, como sugiere esta introducción, la técnica de bootstrapping es útil para prácticamente **cualquier estadístico**, revisaremos su aplicación con la media, es decir cuando $\theta = \mu$ y $\hat{\theta} = \bar{x}$. Queda pendiente (como ejercicio propuesto) aplicarla a proporciones ($\theta = p$ y $\hat{\theta} = \hat{p}$).

12.1.2 Bootstrapping para una muestra

Supongamos que la investigadora Helen Chufe desea evaluar un nuevo algoritmo de clasificación y determinar el tiempo promedio de ejecución (en milisegundos) para instancias de tamaño fijo del problema. Para ello ha realizado pruebas con 10 instancias del problema y registrado los tiempos de ejecución, presentados en la tabla 12.1. La figura 12.1 muestra la distribución del tiempo de ejecución para la muestra.

Instancia		1	2	3	4	5	6	7	8	9	10
Tiempo [ms]		79	75	84	75	94	82	76	90	79	88

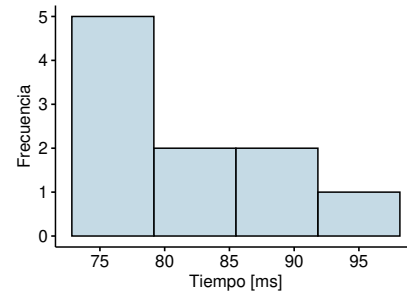


Tabla 12.1: tiempo de ejecución para cada instancia de la muestra.

Figura 12.1: distribución del tiempo de ejecución para la muestra ejemplo.

Evidentemente, la muestra es pequeña ($n = 10$) y su distribución exhibe asimetría hacia la derecha, por lo que Chufe ha decidido emplear bootstrapping como alternativa para enfrentar estos datos problemáticos.

Para ilustrar el proceso paso a paso, consideremos inicialmente $R = 10$ remuestreos y calculemos la media para cada uno. La tabla 12.2 presenta en cada columna una de la muestra y las remuestras obtenidas, con sus respectivas medias en la última fila.

Original	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
79	90	88	90	90	79	79	79	79	79	76
75	84	76	76	84	79	82	79	75	90	79
84	94	76	94	82	79	76	84	84	75	88
75	88	94	84	79	75	79	90	94	88	84
94	82	79	84	90	84	76	94	94	79	88
82	82	82	94	88	94	84	76	79	90	94
76	75	90	75	82	75	82	94	82	75	82
90	90	79	75	76	79	90	79	84	90	82
79	84	79	79	76	90	79	82	79	79	75
88	79	75	84	75	79	75	88	82	79	94
82,2	84,8	81,8	83,5	82,2	81,3	80,2	84,5	83,2	82,4	84,2

Tabla 12.2: muestra original y remuestreos de bootstrap

La figura 12.2 muestra la distribución bootstrap de la media para los 10 remuestreos del ejemplo (figura 12.2a) y para 2.000 remuestreos (figura 12.2b). Vemos que al aumentar la cantidad de remuestras, la forma de la distribución bootstrap se asemeja más a una normal. Es más, esta distribución **converge** a la **distribución muestral teórica** del estadístico, para un tamaño de muestra fijo, a medida de R **aumenta**.

En la tabla 12.2 vemos que $\bar{x} = 82,2$, y que el promedio bootstrap es:

$$\bar{x}_{(\bar{x}, R)} = \frac{1}{10} \sum_{i=1}^{10} \bar{x}_i^* = \frac{84,8 + 81,8 + 83,5 + 82,2 + 81,3 + 80,2 + 84,5 + 83,2 + 82,4 + 84,2}{10} = 82,81$$

La figura 12.2b también nos da una idea acerca de la variabilidad de los promedios de las diferentes remuestras que la muestra original genera. Para el ejemplo con 10 remuestreos, la suma de las desviaciones cuadradas es:

$$\begin{aligned} \sum_{i=1}^{10} (\bar{x}_i^* - 82,81)^2 &= (84,8 - 82,81)^2 + (81,8 - 82,81)^2 + (83,5 - 82,81)^2 + \\ &\quad (82,2 - 82,81)^2 + (81,3 - 82,81)^2 + (80,2 - 82,81)^2 + \\ &\quad (84,5 - 82,81)^2 + (83,2 - 82,81)^2 + (82,4 - 82,81)^2 + \\ &\quad (84,2 - 82,81)^2 \\ &= 20,029 \end{aligned}$$

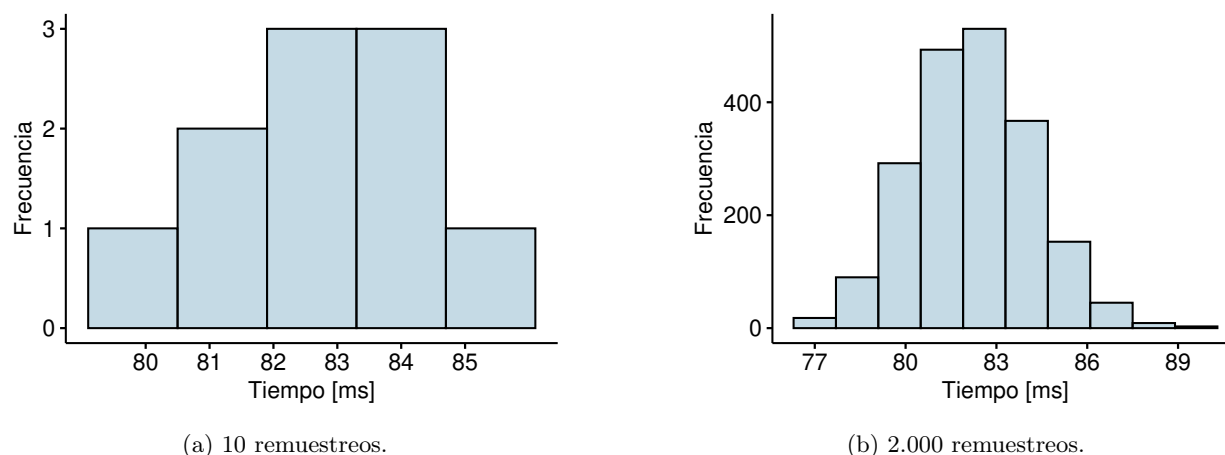


Figura 12.2: distribuciones bootstrap de la media.

En consecuencia, el error estándar de la distribución bootstrap del ejemplo es:

$$SE_{(\bar{x}^*, 10)} = \sqrt{\frac{20,029}{10 - 1}} = 2,225$$

el sesgo es:

$$\delta_{(\bar{x}^*, 10)} = \bar{x}_{(\bar{x}^*, 10)} - \bar{x} = 82,81 - 82,20 = 0,61$$

Ahora que ya conocemos toda esta información de la distribución bootstrap para la media, podemos **construir un intervalo de confianza para la media de la población**, para lo que abordaremos diferentes alternativas.

Cuando la distribución bootstrap se asemeja a la normal y el sesgo es pequeño en comparación con el estimador calculado (como en este caso, ya que $0,61 \ll 82,20$), podemos construir un intervalo de confianza del mismo modo que hicimos en el capítulo 4, teniendo el cuidado de corregir el sesgo detectado, como muestra la ecuación 12.4, donde z^* es el valor crítico para el nivel de confianza requerido.

$$(\bar{x} - \delta_{(\bar{x}^*, 10)}) \pm z^* SE_{(\bar{x}^*, 10)} \quad (12.4)$$

Así, si consideramos para este ejemplo un nivel de significación $\alpha = 0,01$, el valor crítico (bilateral) es $z^* = z_{(1-\alpha/2)} = 2,576$. En consecuencia, el intervalo de 99 % confianza resultante para la media de la población es $[75,859; 87,321]$.

Otra alternativa cuando la distribución bootstrap se asemeja a la normal, y que tiene en cuenta posibles asimetrías, es construir el intervalo de confianza en base a cuantiles críticos. En este caso, para $\alpha = 0,01$, los límites del intervalo están dados por los percentiles 1 y 99 de la distribución bootstrap, que para el ejemplo son: $[80,250; 84,786]$.

Cuando los intervalos de confianza obtenidos por ambos métodos son muy diferentes, es clara señal de que no podemos asumir que la distribución bootstrap se asemeja a la normal. En general, lo más recomendable es usar otro esquema, llamado **BCa** (del inglés *bias-corrected accelerated*), es decir, con sesgo corregido y acelerado. No se detalla aquí el procedimiento, pues requiere el empleo de software.

Desde luego, es inviable usar bootstrapping sin software. En R existen varias funciones para “replicar” operaciones sobre vectores o matrices, por lo que implementar bootstrapping, y remuestreo en general, no se dificulta demasiado. Pero además existen muchos paquetes que implementan numerosas funciones *wrapper* para entregar interfaces específicas, en teoría, más simples para usos comunes. Uno de los más usados es el paquete `boot`, que ofrece las funciones `boot(data, statistic, R)` para generar la distribución bootstrap y `boot.ci(boot.out, conf, type)` para calcular los intervalos de confianza, donde:

- `data`: el conjunto de datos. En caso de matrices y `data.frame`, se considera cada fila como una observación con múltiples variables (columnas).
- `statistic`: función que se aplica a los datos y devuelve un vector con el (o los) estadístico(s) de interés.
- `R`: cantidad de remuestreos bootstrap.
- `boot.out`: objeto de clase `boot`, generado por la función `boot()`.
- `conf`: nivel de confianza ($1 - \alpha$).
- `type`: string o vector que indica los tipos de intervalo de confianza a construir ("`norm`" para el basado en la distribución Z, "`perc`" para el basado en los percentiles y "`bca`" para el método BCa).

Debemos mencionar que la función `boot()` puede recibir otros muchos argumentos, algunos de los cuales veremos más adelante, mientras que otros escapan al alcance de los contenidos aquí expuestos.

El script 12.1 construye intervalos de confianza mediante bootstrapping para el ejemplo, con $R = 1.999$ y manteniendo el nivel de significación $\alpha = 0,01$. En las líneas 15–16 se construye la función para calcular el estadístico de interés (en este caso el promedio), que luego usa la función `boot()` para generar la distribución bootstrap (línea 21), luego de fijar una semilla para la generación de números pseudoaleatorios (línea 20) de manera de garantizar la reproducibilidad de los resultados.

Script 12.1: obtención de intervalos de confianza para la media poblacional mediante bootstrapping.

```

1 library(boot)
2 library(broom)
3 library(dplyr)
4 library(purrr)
5 library(tibble)
6
7 # Crear muestra inicial
8 muestra <- c(79, 75, 84, 75, 94, 82, 76, 90, 79, 88)
9
10 # Establecer cantidad de remuestreos y nivel de significación
11 R <- 1999
12 alfa <- 0.01
13
14 # Función para calcular el estadístico: media de la remuestra
15 media <- function(valores, i) {
16   mean(valores[i])
17 }
18
19 # Construir la distribución bootstrap usando el paquete boot
20 set.seed(432)
21 dist_bt <- boot(muestra, statistic = media, R = R)
22
23 # Mostrar y graficar la distribución bootstrap
24 print(dist_bt)
25 plot(dist_bt)
26
27 # Construir y mostrar los intervalos de confianza
28 ic_bt <- boot.ci(dist_bt, conf = 1 - alfa, type = c("norm", "perc", "bca"))
29 cat("\n\n")
30 print(ic_bt)

```

Es importante revisar la distribución bootstrap para confirmar que tiene la forma esperada. La línea 25 del script 12.1 construye un histograma y un gráfico Q-Q de la distribución de los promedios obtenidos con las remuestras, el que es presentado en la figura 12.3. Podemos notar que la distribución bootstrap obtenida sigue un comportamiento aproximadamente normal, aunque con una pequeña asimetría positiva.

En la línea 28 se muestra el uso de `boot.ci()` para construir los intervalos de confianza con los métodos vistos anteriormente. La figura 12.4 muestra información sobre la distribución bootstrap de promedios, que imprime la línea 24, y los intervalos de confianza obtenidos (línea 30). En ella, vemos que bajo el título **ORDINARY NONPARAMETRIC BOOTSTRAP** hay dos subtítulos: **Call**, con la llamada a la función `boot()` realizada; y **Bootstrap Statistics**, donde aparece la media observada en la muestra (**original**), el sesgo (**bias**) y error estándar de la distribución bootstrap (**std.error**), que resultaron 82,2; 0,061 y 1,989, respectivamente.

Bajo el título `BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS` en la figura 12.4 se encuentran dos subtítulos: `CALL`, con la llamada a la función `boot.ci()` realizada; y luego `Intervals`, donde se muestran los intervalos de confianza obtenidos, que resultaron ser $[77,02; 87,26]$ usando aproximación normal, $[77,60; 87,70]$ usando percentiles y $[77,70; 87,95]$ al usar el método BCa. Considerando estos resultados, con intervalos de confianza parecidos, podemos concluir que tenemos 99% confianza de que el algoritmo tarda entre 77,60 ms y 87,70 ms en ejecutar las instancias del tamaño seleccionado.

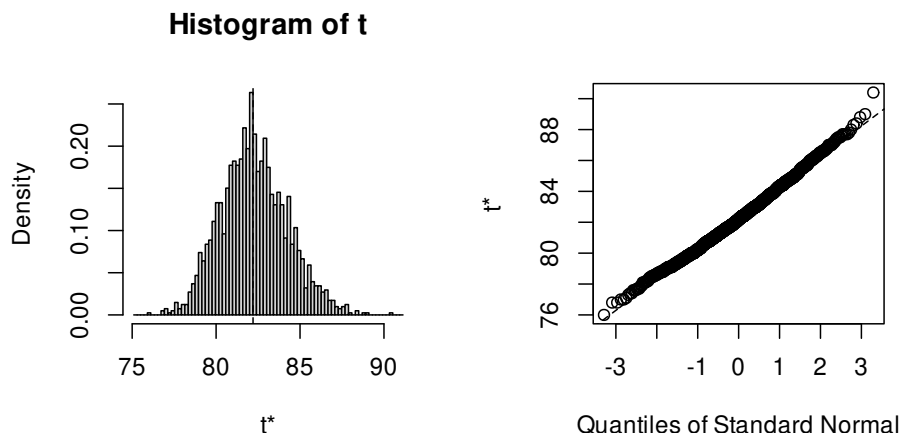


Figura 12.3: histograma y gráfico Q-Q de la distribución bootstrap generada para el ejemplo usando la función `boot()`.

```
ORDINARY NONPARAMETRIC BOOTSTRAP

Call:
boot(data = muestra, statistic = media, R = R)

Bootstrap Statistics :
      original    bias      std. error
t1*      82.2 0.06103052      1.989008

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 1999 bootstrap replicates

CALL :
boot.ci(boot.out = dist_bt, conf = 1 - alfa, type = c("norm", "perc", "bca"))

Intervals :
Level      Normal              Percentile              BCa
99%   (77.02, 87.26 )   (77.60, 87.70 )   (77.70, 87.95 )
Calculations and Intervals on Original Scale
Some percentile intervals may be unstable
Some BCa intervals may be unstable
```

Figura 12.4: estadísticas de la distribución bootstrap obtenida para el ejemplo y los intervalos de confianza calculados a partir de ella.

Vamos aprovechar la oportunidad para introducir algunas ideas de la colección de paquetes `tidyverse` para “modernizar” un poco estos reportes.

En las líneas 33–36 se usa un *pipeline* para obtener las estadísticas de la distribución bootstrap obtenida en el ejemplo. Esta comienza usando la función `tidy()` del paquete `broom` que convierte objetos estadísticos devueltos por funciones base de R en una matriz de datos “limpia y ordenada” (es decir, *tidy*) de clase `tbl_df` (aunque en pantalla aparece como `tibble`). El segundo paso renombra las columnas de la matriz de datos a los conceptos que hemos estudiado: media observada, sesgo y error estándar.

El siguiente paso agrega la columna `Media_bt` con el valor promedio de las medias de las remuestras, aunque en vez de calcularlas como `mean(distribucion_b[["t"]])`, la obtenemos usando la ecuación 12.3 sumando el sesgo a la media observada. El último paso del flujo recorre y cambia las columnas numéricas de la matriz (`mutate(across(where(is.numeric), ...)` aplicándoles una función que les indica que cuando se muestren por pantalla lo hagan usando 4 decimales (`... ~ num(.x, digits = 4)`). Esta última expresión utiliza una de las notaciones disponibles en `purrr` para funciones anónimas, que es equivalente a: `function(.x) { num(.x, digits = 4) }`.

De manera similar, las líneas 40–44 presentan un *pipeline* para obtener los intervalos de confianza usando bootstrapping. Aquí hacemos uso de una versión de la función `map()` del paquete `purrr`, diseñadas para aplicar de manera “limpia y ordenada” una función a cada elemento de una lista o vector, reemplazando funciones base como `apply()`, `sapply()`, `lapply()`, `mapply()`, etc. Específicamente, usamos la función `map_df()` que espera que la función aplicada devuelva una matriz de datos (`data.frame` o `tibble`) en cada paso, las que luego une por filas para devolver un único objeto `tbl_df` con los resultados. Notemos que en este paso usamos la misma función `tidy()` que vimos anteriormente, pero esta vez indicamos que queremos que devuelva un intervalo de confianza (`conf.int = TRUE`) al nivel $1 - \alpha$ (`conf.level = 1 - alfa`), que es aplicada a cada uno de los tipos de intervalo de confianza definidos en el vector `Tipo_IC` en la línea 39 del script. Así, `.x` toma los valores `"norm"`, `"perc"` y `"bca"`, respectivamente en cada iteración, para luego unir los intervalos de confianza devueltos por la función `tidy()` en cada caso (en la forma de un `tibble` de una fila) en una única matriz de datos, que es retornada al flujo.

El resto del *pipeline* es similar al anterior: selecciona las columnas relevantes (los límites de los intervalos, paso 2), las renombra con etiquetas en castellano (paso 3), define que se muestren con 4 dígitos decimales (paso 4) y agrega una columna con los tipos de intervalos obtenidos (paso 5). Este último paso muestra otra forma de agregar una columna a una matriz de datos *tidy*, a través de la función `add_column()` en vez de `mutate()`, que es más conveniente cuando los datos no provienen de la operación de otras columnas de la matriz.

Script 12.2: (continuación del script 12.1) obtención de estadísticas, intervalos de confianza y valor *p* basados en bootstrapping usando funciones *tidy*.

```
32 # Obtener estadísticas de la distribución bootstrap usando tidyverse
33 dist_bt_stats <- tidy(dist_bt) |>
34   rename(Media_obs = statistic, Sesgo = bias, Error_est = std.error) |>
35   mutate(Media_bt = Media_obs + Sesgo) |>
36   mutate(across(where(is.numeric), ~ num(.x, digits = 4)))
37
38 # Obtener intervalos de confianza usando tidyverse
39 Tipo_IC <- c("norm", "perc", "bca")
40 ic_bt <- map_df(Tipo_IC, ~ tidy(dist_bt, conf.int = TRUE, conf.level = 1 - alfa,
41                               conf.method = .x)) |>
42   select(conf.low, conf.high) |> rename(Lim_inf = conf.low, Lim_sup = conf.high) |>
43   mutate(across(where(is.numeric), ~ num(., digits = 4))) |>
44   add_column(Tipo_IC, .before = 1)
45
46 # Mostrar estadísticas e intervalos de confianza con un look más moderno
47 msg <- "Estadísticas bootstrap - una muestra"
48 cat("\n\n", msg, ":\n", sep = "")
49 cat(rep("-", nchar(msg)), "\n", sep = "")
50 print(dist_bt_stats)
51
52 msg <- "Intervalos de confianza para la media poblacional"
53 cat("\n", msg, ":\n", sep = "")
54 cat(rep("-", nchar(msg)), "\n", sep = "")
55 print(ic_bt)
56
57 # Desplazar la distribución bootstrap para que se centre en el valor nulo
58 valor_nulo <- 75
59 desplazamiento <- dist_bt_stats["Media_bt"] - valor_nulo
60 dist_nula_bt <- dist_bt[["t"]] - desplazamiento
61
62 # Determinar y mostrar el valor p bootstrap
63 p <- (sum(dist_nula_bt > dist_bt_stats["Media_obs"]) + 1) / (R + 1)
64 cat("\nValor p*: ", sprintf("%.3f", p), "\n")
```



```

Estadísticas bootstrap - una muestra:
-----
# A tibble: 1 × 4
  Media_obs      Sesgo Error_est  Media_bt
  <num:.4!> <num:.4!> <num:.4!> <num:.4!>
1   82.2000    0.0610    1.9890    82.2610

Intervalos de confianza para la media poblacional:
-----
# A tibble: 3 × 3
  Tipo_IC      Lim_inf      Lim_sup
  <chr>      <num:.4!> <num:.4!>
1 norm      77.0156    87.2623
2 perc      77.6000    87.7000
3 bca       77.7000    87.9534

Valor p*: 0.001

```

Figura 12.5: estadísticas, intervalos de confianza y valor p basados en bootstrapping para el ejemplo obtenidas con funciones *tidy*.

La figura 12.5 muestra los resultados que aparecen en la consola al ejecutar el script 12.2. La primera tabla, desplegada en las líneas 47–50, corresponde a los estadísticos de la distribución bootstrap, y la segunda, que es mostrada en las líneas 52–55, a los intervalos de confianza obtenidos. Podemos apreciar que es la misma información vista en la figura 12.4, a la que hemos agregado explícitamente la media de la distribución bootstrap (**Media_bt**), pero ahora la tenemos en matrices de datos que podemos usar más fácilmente en futuras operaciones.

Supongamos ahora que Helen desea realizar el procedimiento usual de contraste de hipótesis para determinar si el tiempo promedio de ejecución del algoritmo para instancias del tamaño seleccionado es mayor a 75 milisegundos. Así, tenemos que:

H_0 : el tiempo medio (μ) que tarda el nuevo algoritmo de clasificación para resolver instancias de tamaño fijo del problema es de 75 milisegundos. Matemáticamente: $\mu = 75$ [ms]

H_A : el nuevo algoritmo de clasificación requiere, en promedio, más de 75 milisegundos para resolver las instancias de tamaño fijo del problema en estudio. Matemáticamente: $\mu > 75$ [ms]

El contraste de hipótesis clásico se basa en una **distribución muestral centrada en el valor nulo** para, a partir de ella, obtener el valor p. Sabemos que la distribución bootstrap se centra alrededor del valor observado, por lo que debemos **desplazarla** para que represente la hipótesis nula. Para lograrlo, simplemente necesitamos restar a cada observación de la distribución bootstrap la **diferencia** entre su valor promedio y el valor nulo.

En general, cuando usamos remuestreo el valor p queda dado por la fórmula señalada en la ecuación 12.5:

$$p = \frac{r + 1}{R + 1} \quad (12.5)$$

donde:

- r : cantidad de valores en la **distribución nula remuestreada** (que representa H_0) que son **a lo menos tan extremas como el estadístico observado**.
- R : cantidad de remuestras consideradas en la simulación.

Es decir, cuando la hipótesis alternativa es que el estadístico es “menor” (o “a lo más”) que el valor nulo, r corresponde a $\text{contar}(\hat{\theta}_i^* \leq \hat{\theta})$; al contrario, cuando la hipótesis alternativa es que el estadístico es “mayor” (o “a lo menos”) que el valor nulo, r es $\text{contar}(\hat{\theta}_i^* \geq \hat{\theta})$; finalmente, cuando la hipótesis alternativa es que el estadístico es “distinto” al valor nulo (bilateral), r se obtiene como $\text{contar}(|\hat{\theta}_i^*| \geq |\hat{\theta}|)$. Además, debemos notar que en el cálculo del valor p también se considera el valor observado en la muestra original, por lo que se suma 1 en el conteo y en el número de repeticiones.

El script 12.2 también muestra cómo aplicar estas ideas al ejemplo. En las líneas 58–60 se desplaza la distribución bootstrap para reflejar la hipótesis nula, centrando la distribución en torno al valor nulo (75,0);

luego, en la línea 63, se calcula el valor p considerando todas las observaciones iguales o mayores al valor observado 82,2; que es mostrado en pantalla por la línea 64. Este resultado puede verse en la última línea de la figura 12.5, que nos indica que $p^* = 0,001$; menor que el nivel de significación, por lo que la evidencia es suficientemente fuerte para rechazar la hipótesis nula en favor de la hipótesis alternativa. En consecuencia, concluimos con 99 % confianza que el tiempo de ejecución promedio del algoritmo para instancias del tamaño seleccionado supera los 75 milisegundos.

Una observación importante: a este resultado significativo **no podemos asociarle** el intervalo de confianza bootstrap obtenido ([77,60; 87,70]), puesto que el valor p^* se obtuvo para una hipótesis alternativa unilateral ($\mu > 75$ [ms]) mientras que el intervalo se calculó de manera bilateral.

12.1.3 Bootstrapping para dos muestras independientes

El proceso para comparar dos poblaciones mediante bootstrapping es similar al que ya conocimos para una única población, teniendo cuidado de remuestrear separadamente de cada una de las muestras para mantener su independencia y sus tamaños. Entonces, si tenemos dos muestras independientes M_A y M_B , potencialmente provenientes de dos poblaciones diferentes, de tamaños n_A y n_B respectivamente, los pasos a seguir son:

1. Fijar la cantidad R de repeticiones bootstrap.
2. En cada repetición:
 - a) hacer un remuestreo con reposición de tamaño n_A a partir de la muestra M_A .
 - b) hacer un remuestreo con reposición de tamaño n_B a partir de la muestra M_B .
 - c) calcular el estadístico de interés con las remuestras conseguidas
3. Construir el intervalo de confianza para el estadístico de interés a partir de la distribución bootstrap generada.

Si bien la función `boot()` vista anteriormente puede manejar perfectamente este tipo de remuestreo, vamos a seguir el impulso de trabajar con la filosofía *tidy* que introdujo la colección de paquetes `tidyverse`.

Para ejemplificar, supongamos que una Universidad desea confirmar que, en promedio, las calificaciones finales de las estudiantes mujeres es superior a la de los hombres que rinden una asignatura inicial de programación por primera vez. Para ello, disponen de las notas (en escala de 1,0 a 7,0) de 15 hombres y 13 mujeres:

Hombres: 4,1; 5,9; 4,4; 3,4; 3,6; 3,1; 3,4; 4,0; 3,7; 3,8; 5,9; 3,6; 4,5; 3,8; 4,2

Mujeres: 3,7; 5,4; 5,3; 4,9; 5,7; 5,1; 5,9; 5,6; 4,8; 5,4; 5,9; 5,3; 3,2

El script 12.3 lista el comienzo del análisis de este ejemplo en R.

Script 12.3: revisión de la condición de normalidad de dos muestras independientes.

```

1 library(ggpubr)
2 library(glue)
3 library(rsample)
4 library(rstatix)
5 library(tidyverse) # dplyr, ggplot2, purrr, tibble, tidyr
6
7 # Definir las muestras obtenidas
8 notas_h <- c(4.1, 5.9, 4.4, 3.4, 3.6, 3.1, 3.4, 4.0, 3.7, 3.8, 5.9, 3.6, 4.5, 3.8, 4.2)
9 notas_m <- c(3.7, 5.4, 5.3, 4.9, 5.7, 5.1, 5.9, 5.6, 4.8, 5.4, 5.9, 5.3, 3.2)
10
11 # Crear matriz de datos con las muestras independientes
12 Muestra <- factor(c(rep("hombres", length(notas_h)), rep("mujeres", length(notas_m))))
13 datos_largos <- tibble(Muestra, Nota = c(notas_h, notas_m))
14
15 # Comprobar la normalidad de las muestras
16 norm_tests <- datos_largos |> group_by(Muestra) |> shapiro_test(Nota) |>
17   select(-variable) |> rename(W = statistic, Valor_p = p)
18
19 msg <- "Pruebas de normalidad"
20 cat(msg, "\n", sep = "")
21 cat(rep("-", nchar(msg)), "\n", sep = "")
22 print(norm_tests)

```

Este código tiene la finalidad de definir los datos y evaluar la normalidad de las muestras independientes. Primero, observamos que hemos incluido la colección completa de paquetes `tidyverse` (línea 5), aunque esto no es una buena práctica pues se podrían importar muchas funciones que no se necesitan; peor aún, si no somos cuidadosos podríamos sobrescribir alguna que sí deseamos utilizar. De hecho, al comenzar el script 12.3, R nos advierte: `dplyr::filter()` masks `rstatix::filter()`, `stats::filter()`; es decir, una llamada a la función `filter()` se interpreta como una invocación a la función que implementa el paquete `dplyr`, que probablemente es lo que queremos, pero si importáramos los paquetes en otro orden, esta implementación se ocultaría y necesitaríamos ser explícitos al llamarla, y escribir `dplyr::filter()` (en vez de simplemente `filter()`).

El paquete `rstatix` proporciona implementaciones para reemplazar las funciones base de análisis estadísticos comunes de una manera *tidy*, facilitando su integración con la colección `tidyverse`. En este script, usamos la función `shapiro_test()` en el *pipeline* de las líneas 16–17, para aplicar esta prueba de normalidad a los datos previamente agrupados por los dos niveles del factor `Muestra`, usando la función `group_by()` del paquete `dplyr`. En los siguientes pasos, los resultados de estas pruebas auxiliares, en la forma de una matriz de datos `tibble`, se ordenan eliminando una columna innecesaria y renombrando las columnas para darle una mayor legibilidad. La tabla resultante es desplegada en pantalla, junto a un mensaje, en las líneas 19–22 del script 12.3, como muestra la figura 12.6.

Pruebas de normalidad:

```
-----
# A tibble: 2 × 3
  Muestra      W Valor_p
  <fct>    <dbl>   <dbl>
1 hombres 0.830 0.00923
2 mujeres 0.831 0.0161
```

Figura 12.6: resultado de las pruebas de normalidad aplicadas por el script 12.3 para el ejemplo.

Tras revisar estos resultados de las pruebas de Shapiro-Wilk, el equipo de investigación ha determinado que ni las notas de los hombres ($W = 0,830$; $p = 0,009$) ni las de las mujeres ($W = 0,831$, $p = 0,016$) siguen una distribución normal considerando un nivel de significación $\alpha = 0,05$. Por esta razón, el equipo ha decidido usar bootstrapping para la prueba de hipótesis, usando $R = 1.999$ repeticiones.

Para cumplir con este requerimiento haremos uso del paquete `rsample`, que pertenece a la colección de paquetes `tidymodels` que proporciona un *framework* para crear modelos estadísticos y aplicar técnicas de aprendizaje automático (*machine learning*) siguiendo los principios de `tidyverse`.

El script 12.4 muestra la continuación del ejemplo, que comienza por obtener las R remuestras requeridas por medio de la función `bootstraps()` en la línea 27. Notemos que previamente fijamos una semilla (línea 26) para asegurarnos que cada vez que se ejecute el script se generen las mismas remuestras. También que usamos el argumento `apparent = TRUE` para indicar a la función que conserve la muestra original en la lista de remuestras que devuelve.

Script 12.4: (continuación del script 12.3) obtención de una distribución bootstrap para la diferencia de dos medias del ejemplo.

```
24 # Crear las remuestras, incluyendo la muestra original
25 R <- 1999
26 set.seed(432)
27 remuestras <- bootstraps(datos_largos, times = R, strata = Muestra, apparent = TRUE)
28
29 # Función que devuelve los estadísticos de interés para la diferencia
30 # de las medias de dos muestras independientes en una remuestra.
31 dif_dos_medias <- function(remuestra, ...) {
32   analysis(remuestra) |> group_by(Muestra) |>
33     summarise(n = n(), m = mean(Nota), v = var(Nota)) |>
34     summarise(term = "Dif. 2 medias", estimate = m[1] - m[2], std.error = sqrt(sum(v / n)))
35 }
36
37 # Obtener la distribución bootstrap de la diferencia de las medias
38 dist_bt <- remuestras |> mutate(stats = map(splits, dif_dos_medias))
```

Más importante, en este caso, es notar que usamos el argumento `strata = Muestra` para indicar que el muestreo con reposición debe hacerse de manera independiente para **cada estrato** (esto es, cada grupo) que define (cada nivel de) el factor `Muestra`. Recordemos que en el ejemplo se tienen dos grupos: `hombres` y `mujeres`, como apreciamos en la figura 12.6.

Luego, en las líneas 31–35 del script 12.4, definimos la función `dif_dos_medias(remuestra, ...)` que calcula el estadístico de interés. Fijémonos en que la función recibe una remuestra como primer argumento, que ha de corresponder a un objeto `rsplit`, que es una representación eficiente para remuestras que utilizan las funciones del paquete `rsample`; mientras que el segundo argumento es una lista opcional de argumentos (`...`). A continuación, observemos que el cuerpo de la función corresponde a un único *pipeline* que comienza convirtiendo la remuestra en la matriz de datos *tidy* correspondiente, que luego agrupa por el factor `Muestra`, por lo que en el tercer paso se calcula el número de observaciones, la media y la varianza de cada grupo. El último paso construye una matriz de datos con tres variables: `term`, con la descripción del estadístico calculado; `estimate`, con el valor del estadístico para esa remuestra; y `std.error` con el error estándar del estadístico en esa remuestra. Si bien esta función puede simplemente devolver la estimación del estadístico, hemos optado por esta forma más compleja por razones que expondremos más adelante. Otro aspecto importante es que, a cambio de la elegancia de este *pipeline*, esta función es ¡más de 10 veces más lenta! que una implementación con funciones base que devuelva un objeto `data.frame`, que cumpliría el mismo propósito, y ¡más de 30 veces más lenta! si además se usa la función `boot()` en vez del paquete `rsample`. Esto debe tenerse en mente si alguna vez necesitamos ejecutar decenas o cientos de miles de iteraciones de bootstrap.

La línea 38 del script aplica la función `dif_dos_medias(remuestra, ...)` a cada una de las remuestras generadas en la línea 27, incluyendo la muestra original, cuyo resultado es guardado en una nueva columna, llamada `stats`.

Otra desventaja es que el paquete `rsample` no entrega estadísticas ni gráficas de la distribución bootstrap obtenida, por lo que debemos implementarlo nosotros como muestra el script 12.5, que comienza uniendo la lista de matrices de datos con los estadísticos obtenidos para cada remuestra en una sola tabla por medio de la función `unnest()` del paquete `tidyr`.

Script 12.5: (continuación del script 12.4) estadísticas y gráficos para la distribución bootstrap de la diferencia de dos medias obtenida para el ejemplo.

```
40 # Simplificar la estructura de la distribución bootstrap a una tabla para su análisis
41 dist_bt_ttbl <- dist_bt |> select(-splits) |> unnest(cols = stats)
42
43 # Obtener y mostrar las estadísticas de la distribución bootstrap
44 dif_obs <- dist_bt_ttbl |> filter(id == "Apparent") |> pull(estimate)
45 dist_bt_stats <- dist_bt_ttbl |> filter(id != "Apparent") |>
46   summarise(Dif_obs = dif_obs, Dif_bt = mean(estimate), Dif_bt_es = sd(estimate)) |>
47   mutate(Sesgo = Dif_bt - Dif_obs, .before = 3) |> map_df(~ num(.x, digits = 4))
48
49 msg <- "Estadísticas de la distribución bootstrap - diferencia de dos medias"
50 cat("\n\n", msg, ":\n", sep = "")
51 cat(rep("-", nchar(msg)), "\n", sep = "")
52 print(dist_bt_stats)
53
54 # Crear y mostrar los gráficos para la distribución bootstrap
55 dist_bt_his <- dist_bt_ttbl |> filter(id != "Apparent") |>
56   gghistogram(x = "estimate", bins = nclass.scott(dist_bt_ttbl[["estimate"]]),
57     color = "steelblue", fill = "steelblue",
58     xlab = "Diferencia de Medias", ylab = "Frecuencia") +
59   geom_vline(xintercept = dif_obs, color = "steelblue4",
60     linetype = "dashed", linewidth = 1.2)
61 dist_bt_qq <- dist_bt_ttbl |> filter(id != "Apparent") |>
62   ggqqplot(x = "estimate", color = "steelblue", fill = "steelblue",
63     xlab = "Cuantiles teóricos (Normal)", ylab = "Cuantiles muestrales")
64 titulo <- text_grob("Distribución bootstrap de la diferencia de dos medias",
65   size = 12, face = "bold")
66 dist_bt_pl <- ggarrange(dist_bt_his, dist_bt_qq, ncol = 2, labels = "AUTO") |>
67   annotate_figure(top = titulo)
68 print(dist_bt_pl)
```

En la línea 44 del script 12.5 se obtiene la diferencia de las medias observada en la muestra original, identificada en la matriz de datos con la etiqueta `"Apparent"`. En las siguientes líneas (45–47) se aplica un *pipeline* que primero quita la muestra original de la matriz de datos, para luego resumirla en una tabla de una fila con la diferencia de las medias observada, la media de las diferencias encontradas en las remuestras y el error estándar de estas diferencias. El flujo luego agrega una columna con el sesgo encontrado y finalmente fija que cuando estas columnas se desplieguen, lo hagan presentando cuatro dígitos decimales. El resultado se despliega en las líneas 49–52, que resulta en la salida mostrada en la figura 12.7.

```
Estadísticas de la distribución bootstrap - diferencia de dos medias:
-----
# A tibble: 1 × 4
  Dif_obs   Dif_bt   Sesgo Dif_bt_es
<num:.4!> <num:.4!> <num:.4!> <num:.4!>
1  -0.9990  -1.0047  -0.0057  0.2926
```

Figura 12.7: estadísticas de la distribución bootstrap para la diferencia de dos medias del ejemplo.

La figura 12.8 presenta un histograma y un gráfico cuartil-cuartil, como los que se producen para un objeto `boot`, que se construyen en las líneas 55–68 del script 12.5. Cabe destacar que el número de intervalos del histograma es determinado con el criterio de Scott (Scott, 1979), que utiliza la desviación estándar y el tamaño de la muestra para determinar el ancho óptimo de los intervalos, implementado en la función base `nclass.scott()`. Podemos ver que la distribución bootstrap obtenida, aunque levemente asimétrica, se aproxima a una distribución normal centrada en el valor -1 aproximadamente.

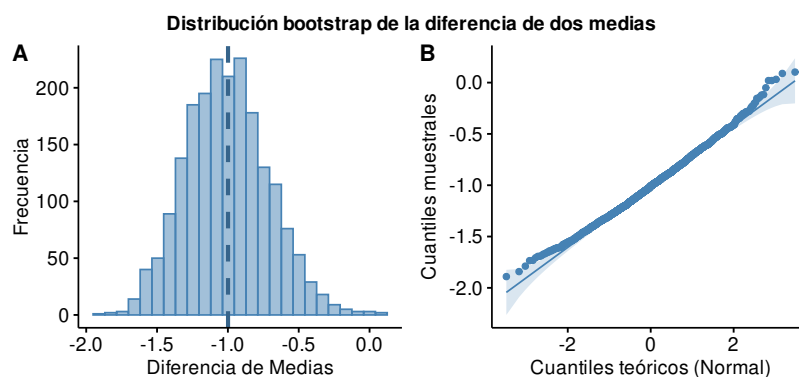


Figura 12.8: histograma (A) y gráfico Q-Q (B) para la distribución bootstrap de la diferencia de dos medias.

Es necesario que obtengamos al menos un intervalo de confianza para avanzar en nuestro análisis. Esto es lo que hace el código mostrado en el script 12.6. Como en el ejemplo se ha detectado una superioridad de las mujeres, es probable que necesitemos **intervalos unilaterales**, pero las funciones para obtenerlos ¡solo estiman bilaterales! Pero existe un truco para conseguir lo que queremos: vamos a solicitar intervalos con el **doble** del nivel de significación, para luego ignorar uno de los extremos.

Script 12.6: (continuación del script 12.5) obtención de intervalos de confianza para la distribución bootstrap de la diferencia de dos medias.

```
70 # Calcular y mostrar intervalos de confianza unilaterales para el estadístico de interés
71 alfa <- 0.05
72 ic_t <- dist_bt |> int_t(statistics = stats, alpha = 2 * alfa)
73 ic_perc <- dist_bt |> int_pctl(statistics = stats, alpha = 2 * alfa)
74 ic_bias <- dist_bt |> int_bca(statistics = stats, alpha = 2 * alfa, .fn = dif_dos_medias)
75 ics <- bind_rows(ic_t, ic_perc, ic_bias) |> select(-.alpha) |>
76   mutate(.lower = -Inf) |> mutate(across(where(is.numeric), ~ num(., digits = 4)))
77
78 msg <- "Intervalos de confianza unilaterales para la diferencia de dos medias"
79 cat("\n", msg, ":\n", sep = "")
80 cat(rep("-", nchar(msg)), "\n", sep = "")
81 print(ics)
```

El script comienza definiendo un nivel de significación, que luego utiliza en las llamadas que obtienen, respectivamente, intervalos de confianza de tipos t (función `int_t()`, línea 72), percentil (`int_pctl()`, línea 73) y BCa (`int_bca()`, línea 74). Ya conocemos los dos últimos tipos de intervalos de confianza; solo mencionar que, como ocurría internamente con la función `boot.ci()`, el cálculo del intervalo de tipo Bca realiza un remuestreo propio, por lo que necesita utilizar la función que calcula el estadístico, la que especificamos en el argumento `.fn`. El intervalo de confianza t, como sugiere el nombre, se calcula bajo el supuesto de que la **distribución muestral** del estadístico sigue una **distribución t**. Si hacemos memoria, para estimar este intervalo necesitábamos calcular el error estándar de la distribución muestral (ecuación 5.2). Es por esta razón que la función `dif_dos_medias()` calcula, además del valor del estadístico, el error estándar observado en cada remuestra bootstrap. Sin estos datos, no es posible determinar un intervalo de confianza t. Debemos decir que la función `boot.ci(type = "stud")` también puede calcular este tipo de intervalo si se le proveen estos datos.

Las funciones `int_t()`, `int_pctl()` y `int_bca()` también explican que la función `dif_dos_medias()` no retorne valores numéricos, pues estas funciones esperan que el argumento `statistics` especifique el nombre de la columna (sin comillas) en el conjunto de datos que contiene una lista de `tibbles` con las estimaciones bootstrap individuales, con columnas `term`, `estimate` y, opcionalmente, `std.error` si se desean intervalos de confianza de tipo t.

Las líneas 75–76 del script 12.6 une los resultados que devuelven estas funciones en un único `tibble`, elimina el límite inferior de los intervalos y ordena la tabla para su despliegue en pantalla, que hace en las líneas 78–81. El resultado se muestra en la figura 12.9.

```
Intervalos de confianza unilaterales para la diferencia de dos medias:
-----
# A tibble: 3 × 5
  term      .lower .estimate .upper .method
<chr>      <num:.4!> <num:.4!> <num:.4!> <chr>
1 Dif. 2 medias -Inf      -1.0047  -0.3315 student-t
2 Dif. 2 medias -Inf      -1.0047  -0.5087 percentile
3 Dif. 2 medias -Inf      -1.0047  -0.4483 BCa
```

Figura 12.9: intervalos de confianza unilaterales para la diferencia de dos medias obtenidas desde la distribución bootstrap.

Vemos que los intervalos de confianza obtenidos son un tanto diferentes, una indicación de que la distribución bootstrap tiene una asimetría que no debemos ignorar, confirmando la impresión visual que tuvimos de la figura 12.8. De esta forma, con 1.999 repeticiones bootstrap y 95 % confianza, hemos determinado que la diferencia en la calificación final entre hombres y mujeres se encuentra en el intervalo BCa $(-\infty; -0,448]$.

Supongamos ahora que las personas responsables del estudio de todas formas desean realizar un contraste de hipótesis, con un nivel de significación $\alpha = 0,05$, de si la diferencia entre las calificaciones finales entre hombres y mujeres es mayor a 5 décimas, en favor de las mujeres. Las hipótesis serían:

H_0 : la media de las calificaciones finales de las mujeres (μ_m) en la asignatura inicial de programación es 5 décimas mayor que la media de los hombres (μ_h), es decir: $\mu_h - \mu_m = -0,5$

H_A : la media de las calificaciones finales de las mujeres en la asignatura inicial de programación supera en más de 5 décimas a la media de los hombres, esto es: $\mu_h - \mu_m < -0,5$

Como vimos en la subsección anterior, la distribución bootstrap con que contamos es útil para obtener intervalos de confianza para el estadístico de interés, pero no es apropiada para estimar un valor p debido a que no refleja necesariamente la hipótesis nula.

También vimos una forma de conseguir una distribución bootstrap que permitiera este análisis, que conseguimos desplazando los valores del estadístico para que la distribución quedara centrada en el valor nulo. Si bien esta alternativa también es válida en este caso, este procedimiento se dificulta cuando se trabaja con parámetros de interés distintos a la media o a la proporción, o con más de dos muestras. Un método más general consiste en realizar el **desplazamiento de las muestras** para que estas reflejen la hipótesis nula **antes de aplicar el procedimiento de remuestreo**.

Script 12.7: (continuación del script 12.6) desplazamiento de una de las muestras del ejemplo para representar la situación bajo la hipótesis nula.

```

83 # Desplazamos los valores de la muestra de hombres para reflejar la hipótesis nula
84 valor_nulo <- -0.5
85 desplazamiento <- valor_nulo - dif_obs
86 datos_largos_H0 <- datos_largos |>
87   mutate(Nota = if_else(Muestra == "hombres", Nota + desplazamiento, Nota))
88
89 # Generamos las mismas remuestras con los datos que cumplen la hipótesis nula
90 set.seed(432)
91 remuestras_H0 <- bootstraps(datos_largos_H0, times = R, strata = Muestra, apparent = TRUE)
92
93 # Solo por esta vez, vamos a verificar que se han usado las mismas observaciones en ambos
94 # conjuntos de remuestras (usan los mismos índices)
95
96 # Primero, obtenemos los índices que usan y las ordenamos como matrices
97 indices_orig <- remuestras |> pull(splits) |> map(~ .x[["in_id"]]) |>
98   set_names(remuestras |> pull(id)) |> as_tibble() |> t()
99 indices_desp <- remuestras_H0 |> pull(splits) |> map(~ .x[["in_id"]]) |>
100   set_names(remuestras_H0 |> pull(id)) |> as_tibble() |> t()
101
102 # Ahora mostramos las primeras filas de cada grupo de remuestras y el resultado final
103 # de la comparación de todas las filas de las matrices.
104 msg <- "Comparación de los índices usados en las remuestras"
105 cat("\n", msg, ":\n", sep = "")
106 cat(rep("-", nchar(msg)), "\n", sep = "")
107 walk(1:5, ~ {
108   cat(sprintf("# i = %d\n", .x));
109   cat("Orig.: ", paste(sprintf("%02d", indices_orig[,.x, ])), "\n");
110   cat("H0     ", paste(sprintf("%02d", indices_desp[,.x, ])), "\n");
111 })
112 cat("\n¿Todas las remuestras son iguales?: ", all(indices_orig == indices_desp))

```

Este método es ilustrado para el ejemplo en el script 12.7. En este caso, la hipótesis nula solo establece que la diferencia entre la calificación media de los hombres y la calificación media de las mujeres es igual a -5 décimas, pero no asume, por ejemplo, la igualdad de las distribuciones subyacentes. En consecuencia, el procedimiento comienza calculando el desplazamiento que debe hacerse a **una de las muestras** en líneas 84–85 del script 12.7, que luego es aplicada a la de los hombres en el *pipeline* de las líneas 86–87.

Luego, generamos las mismas remuestras usadas para el cálculo de los intervalos de confianza (script 12.4), lo que conseguimos fijando la misma semilla (línea 90) y usando los mismos parámetros en la función `bootstraps()` (línea 91), como se comprueba en la figura 12.10.

```

Comparación de los índices usados en las remuestras:
-----
# i = 1
Orig.: 03 04 05 09 09 10 10 10 11 11 11 13 14 15 15 16 16 17 18 20 20 20 22 22 24 25 26 28
H0    : 03 04 05 09 09 10 10 10 11 11 11 13 14 15 15 16 16 17 18 20 20 20 22 22 24 25 26 28
# i = 2
Orig.: 01 02 02 03 03 03 06 06 09 10 12 13 14 14 15 16 16 18 18 21 21 21 23 24 25 25 26 28
H0    : 01 02 02 03 03 03 06 06 09 10 12 13 14 14 15 16 16 18 18 21 21 21 23 24 25 25 26 28
# i = 3
Orig.: 01 03 03 03 05 06 09 09 09 12 12 14 15 15 15 18 18 19 20 22 22 23 26 26 27 27 28 28
H0    : 01 03 03 03 05 06 09 09 09 12 12 14 15 15 15 18 18 19 20 22 22 23 26 26 27 27 28 28
# i = 4
Orig.: 01 04 04 05 08 08 08 08 10 10 11 12 12 13 15 16 19 20 20 21 21 21 22 23 23 26 26 26
H0    : 01 04 04 05 08 08 08 08 10 10 11 12 12 13 15 16 19 20 20 21 21 21 22 23 23 26 26 26
# i = 5
Orig.: 02 04 05 05 06 06 06 07 07 08 13 14 14 14 15 16 16 16 17 17 18 19 20 23 23 23 24 25
H0    : 02 04 05 05 06 06 06 07 07 08 13 14 14 14 15 16 16 16 17 17 18 19 20 23 23 23 24 25

¿Todas las remuestras son iguales?: TRUE

```

Figura 12.10: índices de las observaciones utilizadas en los primeros 5 pares de remuestras generadas con las muestras originales (`Orig`) y las muestras desplazadas (`H0`).

Para obtener la figura 12.10, las líneas 97–100 extraen los índices de las observaciones usadas en ambos casos, que dejan almacenados como matrices de enteros, para luego mostrar en pantalla las primeras 5 filas de cada una en las líneas 104–111. En la figura podemos ver que efectivamente los índices que utilizan estos 5 pares de remuestras son los mismos.

Para esto usamos la función `walk()` del paquete `purrr` que opera de manera análoga a la función `map()` y sus variaciones, que hemos visto anteriormente, con la diferencia que esta no devuelve un valor. De todas formas, para convencer a quienes todavía tienen dudas, la línea 112 del script 12.7 imprime en la consola el resultado global de comparar las matrices de índices de observaciones. En la última línea de la figura 12.10, podemos ver que se confirma que estas son iguales.

Con esta tranquilidad, podemos obtener la distribución bootstrap para las remuestras que representan la hipótesis nula, como se muestra en el *pipeline* de las líneas 116–117 del script 12.8.

Script 12.8: (continuación del script 12.7) gráfica del contraste de las hipótesis del ejemplo.

```
114 # Ahora podemos obtener la distribución bootstrap para la diferencia de dos medias
115 # si la hipótesis nula fuera verdadera.
116 dist_bt_H0 <- remuestras_H0 |> mutate(stats = map(splits, dif_dos_medias)) |>
117   unnest(cols = stats)
118
119 # Calcular el valor p
120 n_extremos <- dist_bt_H0 |> filter(id != "Apparent") |>
121   filter(estimate <= dif_obs) |> summarise(n = n())
122 p_valor <- (n_extremos + 1) / (R + 1)
123
124 # Crear y mostrar situación de manera gráfica
125 titulo <- sprintf("Contraste de hipótesis usando\n%d repeticiones de bootstrapping", R)
126 ancho <- diff(range(dist_bt_H0[["estimate"]])) / nclass.scott(dist_bt_H0[["estimate"]])
127 dist_bt_H0_pl <- dist_bt_H0 |> filter(id != "Apparent") |>
128   mutate(extremo = factor(if_else(estimate <= dif_obs, 1, 0))) |>
129   ggplot(aes(x = estimate)) +
130     geom_histogram(aes(alpha = extremo), boundary = dif_obs, binwidth = ancho,
131                   color = "steelblue4", fill = "steelblue") +
132     labs(title = titulo, x = "Diferencia de dos medias", y = "Frecuencia") +
133     geom_vline(xintercept = dif_obs, colour = "steelblue", linetype = "dashed") +
134     annotate("text", x = dif_obs, y = Inf, vjust = 2, hjust = 1.1,
135            label = glue("italic(p) == '{sprintf('%.3f', p_valor)}'"), parse = TRUE) +
136     scale_alpha_manual(values = c("0" = 0.2, "1" = 1), guide = NULL) +
137     theme_pubr() + theme(plot.title = element_text(hjust = 0.5))
138 print(dist_bt_H0_pl)
```

Luego, en las líneas 120–122, se calcula el valor p de la misma forma vista en la subsección 12.1.2, es decir, determinando la proporción de estadísticos bootstrap igualmente o más extremos que el valor observado originalmente. Por último, en las líneas 125–137, se crea un gráfico que representa la situación encontrada en este contraste de hipótesis, el que es presentado en la figura 12.11.

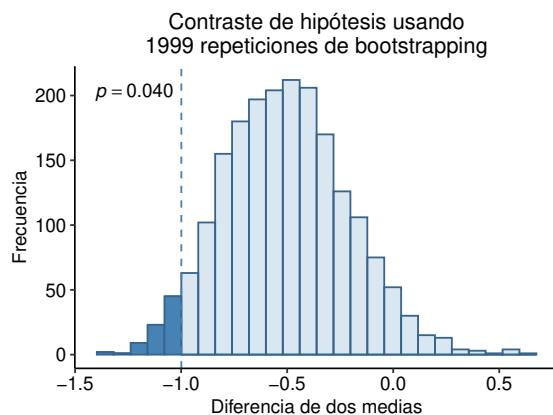


Figura 12.11: contraste de hipótesis usando bootstrapping para la diferencia de dos medias.

En el gráfico podemos ver que la distribución bootstrap de las diferencias de las medias para las remuestras desplazadas está centrada en el valor nulo y que hay una pequeña proporción de valores que son tan o más extremos que la diferencia observada $(-0,999)$ en las muestras originales que estiman un valor $p^* = 0,040$.

Afrontamos entonces una aparente contradicción. Recordemos que obtuvimos que el intervalo de confianza de tipo BCa era $(-\infty; -0,448]$, que **incluye el valor nulo** $-0,5$, sugiriendo que la evidencia no es lo suficientemente fuerte para rechazar la hipótesis nula. Esta contradicción no se daría si hubiéramos ignorado, de forma imprudente, la asimetría de la distribución bootstrap y considerado en su lugar el intervalo de confianza de tipo percentil, $(-\infty; -0,508]$, que deja fuera el valor nulo (por bien poco).

Aquí vale la pena sopesar que el método BCa ajusta los límites del intervalo para corregir el posible error en el intervalo de tipo percentil debido a la asimetría presente en la distribución bootstrap. Por otro lado, el valor p se obtiene con una operación “ingenua” que desplaza la muestra original sumándole una constante, sin preocuparse de la asimetría detectada (no corrige sesgos ni cambios de varianza).

De esta forma, ante esta discrepancia entre el valor p y el intervalo de confianza de tipo BCa, sería recomendable actuar en dos frentes. Primero, aumentar el número de iteraciones bootstrap, $R \geq 5.000$, para reducir el error numérico de la simulación y estabilizar los límites de los intervalos. Segundo, y más importante, incrementar el tamaño de las muestras; esto reduciría la asimetría y el sesgo de la distribución original, permitiendo que el intervalo BCa y el valor p converjan hacia una conclusión más robusta y menos sensible al método de cálculo utilizado. Y si el equipo a cargo del estudio insiste en tomar una decisión basada en el valor p , deberíamos sugerir que el rechazo de la hipótesis nula debe interpretarse con cautela por esta sensibilidad al método de estimación utilizado.

12.1.4 Bootstrapping para dos muestras apareadas

Cuando se tienen dos muestras apareadas, el procedimiento resulta muy sencillo: primero se crea la muestra de la diferencia entre los pares de observaciones, y luego se realiza el procedimiento estudiado para analizar una única muestra que conocimos en la subsección 12.1.2.

Para ejemplificar, supongamos ahora, que la Universidad del ejemplo anterior desea saber si, en promedio, la diferencia entre las calificaciones obtenidas en la primera y la segunda prueba de un curso inicial de programación es de 5 décimas. Para ello, dispone de las siguientes calificaciones (en escala de 1,0 a 7,0) obtenidas en ambas pruebas para una muestra de 20 estudiantes:

Prueba 1: 3,5; 2,7; 1,0; 1,8; 1,6; 4,3; 5,8; 6,4; 3,9; 4,3; 3,4; 5,3; 5,8; 5,3; 2,0; 1,3; 4,0; 5,3; 1,6; 3,6
Prueba 2: 5,2; 5,1; 5,9; 4,8; 1,4; 2,3; 6,8; 5,3; 3,1; 3,8; 4,6; 1,2; 3,9; 2,0; 1,7; 3,3; 6,0; 4,8; 6,9; 1,3

La Universidad ha decidido llevar a cabo el estudio mediante bootstrapping con $R = 3.999$ repeticiones y un nivel de significación $\alpha = 0,05$. Así, el equipo de investigación ha formulado las siguientes hipótesis:

Sean $C_i^{(1)}$ y $C_i^{(2)}$ las calificaciones obtenidas por un/a estudiante de un curso inicial de programación en la primera y la segunda prueba, respectivamente, y $D_i = C_i^{(2)} - C_i^{(1)}$ la diferencia de estas calificaciones. Si μ_D es la media de estas diferencias, entonces:

H_0 : estudiantes del curso inicial de programación suben, en promedio, 5 décimas sus calificaciones en la segunda prueba con respecto a la primera. Matemáticamente: $\mu_D = 0,5$

H_A : en promedio, la diferencia de las calificaciones obtenidas por estudiantes del curso inicial de programación entre la primera y la segunda prueba no es un incremento de 5 décimas. Matemáticamente: $\mu_D \neq 0,5$

Para realizar el análisis inferencial propuesto, el equipo a cargo del estudio ha creado en R el script 12.9, cuya ejecución genera los resultados que se presentan en la figura 12.12, donde se aprecian ciertas diferencias entre los intervalos de tipo percentil y BCa, una indicación de asimetría o varianza inestable en la distribución bootstrap, por lo que consideramos este último. En consecuencia, como el valor nulo está contenido en este intervalo, con 95 % confianza, no es posible descartar que, en promedio, la diferencia de las calificaciones entre la primera y la segunda evaluación de un curso inicial de programación en la Universidad en estudio es de 5 décimas (95 % IC_{BCa}^{*}: $[-0,720; 1.440]$).

Script 12.9: bootstrapping para inferir acerca de la media de las diferencias de dos muestras apareadas.

```

1 library(dplyr)
2 library(purrr)
3 library(rsample)
4 library(tibble)
5 library(tidyr)
6
7 # Crear matriz de datos con las muestras pareadas obtenidas
8 prueba_1 <- c(3.5, 2.7, 1.0, 1.8, 1.6, 4.3, 5.8, 6.4, 3.9, 4.3,
9              3.4, 5.3, 5.8, 5.3, 2.0, 1.3, 4.0, 5.3, 1.6, 3.6)
10 prueba_2 <- c(5.2, 5.1, 5.9, 4.8, 1.4, 2.3, 6.8, 5.3, 3.1, 3.8,
11              4.6, 1.2, 3.9, 2.0, 1.7, 3.3, 6.0, 4.8, 6.9, 1.3)
12 datos <- tibble(Nota = prueba_2 - prueba_1)
13
14 # Crear las remuestras, incluyendo la muestra original
15 R <- 3999
16 set.seed(432)
17 remuestras <- bootstraps(datos, times = R, apparent = TRUE)
18
19 # Función que devuelve la media de una remuestra. En este caso, no
20 # es posible obtener el error estándar de forma analítica.
21 media_dif <- function(remuestra, ...) {
22   analysis(remuestra) |> summarise(term = "Media diferencias", estimate = mean(Nota)) }
23
24 # Obtener la distribución bootstrap de la media de las diferencias
25 dist_bt <- remuestras |> mutate(stats = map(splits, media_dif))
26 dist_bt_tbl <- dist_bt |> select(-splits) |> unnest(cols = stats)
27
28 # Calcular y mostrar los intervalos de confianza para el estadístico de interés
29 alfa <- 0.05
30 ic_perc <- dist_bt |> int_pctl(statistics = stats, alpha = alfa)
31 ic_bias <- dist_bt |> int_bca(statistics = stats, alpha = alfa, .fn = media_dif)
32 ics <- bind_rows(ic_perc, ic_bias) |> select(-.alpha) |>
33   mutate(across(where(is.numeric), ~ num(., digits = 4)))
34
35 msg <- "Intervalos de confianza para la media de las diferencias"
36 cat(msg, ":\n", sep = "")
37 cat(rep("-", nchar(msg)), "\n", sep = "")
38 print(ics)

```

```

Intervalos de confianza para la media de las diferencias:
-----
# A tibble: 2 × 5
  term          .lower .estimate    .upper .method
<chr>          <num:.4!> <num:.4!> <num:.4!> <chr>
1 Media diferencias -0.7350  0.3257  1.4052 percentile
2 Media diferencias -0.7200  0.3257  1.4400 BCa

```

Figura 12.12: intervalos de confianza para la media de las diferencias.

12.1.5 Pruebas de permutación

En el capítulo 8 conocimos la prueba exacta de Fisher, la cual obtiene un valor p exacto tras calcular todas las permutaciones de los datos con iguales valores marginales en una tabla de contingencia y considerar únicamente aquellas permutaciones que ocurren con igual o menor probabilidad que la observada en los datos del estudio. Esta prueba pertenece al grupo conocido como **pruebas exactas de permutación**, cuyo único requisito es la **intercambiabilidad**: si se cumple la hipótesis nula, todas las permutaciones pueden ocurrir con igual probabilidad. En la práctica, este tipo de método puede emplearse para diversos estadísticos que involucren a más de una muestra, tales como la diferencia de proporciones, la diferencia de medias y la razón de varianzas.

Por simpleza y sin pérdida de generalidad, para explicar el método supondremos que queremos inferir sobre la diferencia de un parámetro θ entre dos distribuciones de probabilidad (poblaciones), y que para ello contamos con muestras M_A y M_B de tamaños n_A y n_B , respectivamente. El procedimiento sigue los siguientes pasos:

1. Calcular la diferencia entre el estadístico de interés observado en cada muestra: $\hat{D}_{\text{obs}} = \hat{\theta}_A - \hat{\theta}_B$.
2. Juntar ambas muestras en una muestra combinada: M_{A+B} .
3. Obtener todas las formas $\mathcal{P} = \{M_A^P, M_B^P\}_i$ de separar la muestra combinada M_{A+B} en dos grupos de tamaños n_A y n_B .
4. Construir la distribución de las diferencias en el estadístico de interés desde las muestras en cada una de las permutaciones $\hat{D}_i^P = \{\hat{\theta}_A^P - \hat{\theta}_B^P\}_i$.
5. Calcular el valor p exacto, dado por la proporción de permutaciones en $\mathcal{P}$ cuyo valor (absoluto, si es bilateral) $\hat{D}_i^P$ es menor o mayor al valor (absoluto, si es bilateral) de la diferencia observada $\hat{D}_{\text{obs}}$.

Puesto que las pruebas exactas de permutación requieren calcular todas las permutaciones, solo resultan adecuadas para muestras pequeñas, pues requieren de una enorme cantidad de cálculos. Por ejemplo, para dos muestras con 5 observaciones cada una se deben analizar 252 permutaciones, pero si las muestras fueran de tamaño 20, se tendrían que evaluar ¡más de 137 mil millones de combinaciones!

En consecuencia, si la muestra es grande, suele tomarse una **muestra aleatoria de las posibles permutaciones**, y a partir de ella calcular un **valor p aproximado** utilizando la ecuación 12.5. Las pruebas que siguen este procedimiento se denominan **pruebas (o tests) de permutación** (*permutation tests*, en inglés) o pruebas de aleatorización (*randomization tests*, en inglés) o pruebas de permutación Monte Carlo (Monte Carlo *permutation tests*, en inglés).

En términos generales, el procedimiento para efectuar una prueba de permutación Monte Carlo no es muy distinto al de bootstrapping, aunque hay algunas diferencias fundamentales en el trasfondo:

1. Formular las hipótesis a contrastar e identificar el estadístico de interés θ .
2. Crear una gran cantidad R de permutaciones de las muestras originales¹, usando muestreo sin reposición sobre la muestra combinada.
3. Obtener el estadístico $\hat{\theta}$ para cada permutación, lo que produce una distribución de este estadístico bajo el supuesto de que la hipótesis nula es cierta.
4. Determinar la probabilidad de encontrar en la distribución generada un valor de $\hat{\theta}$ al menos tan extremo como el observado en las muestras originales.

Debemos fijarnos en que, a diferencia de bootstrapping, las pruebas de permutación usan **muestreo sin reposición** puesto que, si la hipótesis nula fuera cierta, cada separación de la muestra combinada sería igualmente probable. Así, lo que se hace en cada repetición es **reordenar** las observaciones y asignarlas ordenadamente a uno de los grupos, respetando los tamaños de las muestras originales.

De los párrafos anteriores debería quedar claro que las pruebas de permutación (exactas o no) solo son **adecuadas para el contraste de hipótesis**, pues determinan una significación estadística (valor p), con **dos o más muestras**. Si solo tenemos una muestra, por más que la permutemos, siempre obtendremos el mismo valor para el estadístico de interés. Por otro lado, como la distribución permutada se construye en base a la condición de intercambiabilidad, esta representa la situación cuando la **hipótesis nula se cumple**, por lo que cualquier intervalo de confianza que se obtenga de ella también representaría esa realidad hipotética.

Para obtener un intervalo de confianza en una prueba de permutación Monte Carlo se emplea la siguiente búsqueda iterativa (siguiendo la notación usada anteriormente):

1. Seleccionar un valor nulo candidato D_0 .
2. Desplazar las muestras para que la hipótesis nula $H_0 : \hat{\theta}_A - \hat{\theta}_B = D_0$ sea cierta.
3. Ejecutar la prueba de permutación Monte Carlo con R repeticiones sobre estos datos desplazados para obtener un valor p aproximado.
4. Repetir este proceso ajustando D_0 , típicamente mediante búsqueda binaria, hasta encontrar los límites inferior y/o superior donde el p-valor obtenido llega al umbral del nivel de significación α fijado.

¹Tradicionalmente se usa un número terminado en 9, pues solían simplificar el cómputo del valor p cuando se hacían manualmente, sin software, utilizando la ecuación 12.5.

Por supuesto este procedimiento es computacionalmente exigente (por cada valor nulo candidato se realizan R permutaciones), y si no se tiene un buen conjunto de candidatos iniciales puede ser extremadamente caro. Una alternativa más práctica es usar bootstrapping para construir un intervalo de confianza si es importante para el análisis, a pesar que los métodos responden a supuestos distintos.

12.1.6 Prueba de permutaciones para dos muestras independientes

Consideremos el siguiente ejemplo: el profesor de una asignatura inicial de programación, que se imparte a estudiantes de primer año de Ingeniería y a estudiantes de último año de otras carreras como electivo, comparó el promedio de calificaciones finales de ambos grupos pues tiene la sensación de que el primero obtiene, en promedio, calificaciones menores a las que consigue el segundo grupo. Para ello, realizó una prueba de permutación con $R = 5.999$ repeticiones utilizando una muestra aleatoria de $n_I = 20$ estudiantes de primer año de Ingeniería y $n_U = 12$ estudiantes de último año de otras carreras. Para su sorpresa, no pudo rechazar la hipótesis nula de igualdad de las medias con un nivel de significación $\alpha = 0,05$, e incluso encontró que ambos grupos tenían prácticamente las mismas calificaciones medias. Intrigado por este resultado, el profesor ha decidido hacer un nuevo estudio con las mismas muestras, comparando ahora la diferencia en la variabilidad (manteniendo la misma cantidad de repeticiones e igual nivel de significación) puesto que sospecha que el grupo de estudiantes de Ingeniería ve aumentado su promedio porque las calificaciones están más esparcidas y hay una compensación por una minoría de calificaciones altas. En consecuencia, decide contrastar las siguientes hipótesis:

H_0 : la varianza de las calificaciones finales de estudiantes de primer año de Ingeniería (σ_I^2) en el curso inicial de programación es la misma que la exhibida por las calificaciones finales de estudiantes de último año de otras carreras en el mismo curso (σ_U^2). Matemáticamente: $\sigma_I^2 - \sigma_U^2 = 0$

H_A : por el contrario, estas varianzas son distintas: $\sigma_I^2 - \sigma_U^2 \neq 0$

Para poder contrastar estas hipótesis, el profesor desarrolló el script 12.10, que primero construye una matriz de datos en formato largo con las muestras conseguidas (líneas 10–16).

Script 12.10: prueba de permutación para la diferencia de dos varianzas.

```

1 library(dplyr)
2 library(ggpubr)
3 library(glove)
4 library(purrr)
5 library(rsample)
6 library(tibble)
7 library(tidyr)
8
9 # Crear muestras iniciales
10 notas_I <- c(5.4, 4.7, 6.3, 2.9, 5.9, 5.1, 2.1, 6.2, 1.6, 6.7, 3.0, 3.3,
11             5.0, 4.1, 3.3, 3.4, 1.2, 3.8, 5.8, 4.2)
12 notas_U <- c(4.0, 4.1, 4.3, 4.3, 4.3, 4.2, 4.3, 4.3, 4.4, 4.1, 4.3, 4.0)
13
14 # Crear matriz de datos con las muestras independientes
15 Muestra <- factor(c(rep("I", length(notas_I)), rep("U", length(notas_U))))
16 datos_largos <- tibble(Muestra, Nota = c(notas_I, notas_U))
17
18 # Crear las remuestras
19 R <- 5999
20 set.seed(432)
21 remuestras <- permutations(datos_largos, permute = Nota, times = R, apparent = TRUE)
22
23 # Función que devuelve los estadísticos de interés para la diferencia de dos varianzas
24 # en una remuestra.
25 dif_dos_varianzas <- function(remuestra, ...) {
26   analysis(remuestra) |> group_by(Muestra) |> summarise(v = var(Nota)) |>
27   summarise(v[1] - v[2]) |> pull()
28 }
29
30 # Obtener la distribución permutada de la diferencia de las medias
31 dist_perm <- remuestras |> mutate(dif_v = map(splits, dif_dos_varianzas)) |> unnest(dif_v)

```

Luego, el script define el número de remuestras deseadas (línea 19), fija una semilla para la generación de números pseudoaleatorios (línea 20) y construye las remuestras con permutación utilizando la función `permutations(data, permute, times, apparent = FALSE)` del paquete `rsample` (línea 21), donde:

- `data` y `apparent`: como vistas para la función `bootstraps()`.
- `permute`: indica la(s) columna(s) de la matriz de datos a permutar en cada resmuestra.
- `times`: cantidad de remuestras a generar (es decir, R).

Notemos que para el ejemplo, en la llamada indicamos que se permute la columna `Nota`. De esta forma, estos valores quedan asignados a los grupos de estudiantes de acuerdo al orden original del factor `Muestra`.

En las líneas 25–28 del script 12.10 se define la función que calcula la diferencia entre las varianzas de ambos grupos como los define la remuestra recibida. Esta función es utilizada en la línea 31 para obtener la distribución permutada. Este *pipeline* ordena los resultados para que estos valores queden en una columna llamada `dif_v`.

Script 12.11: (continuación del script 12.10) gráfica del contraste de las hipótesis de igualdad de las varianzas usando una prueba de permutación.

```

33 # Obtener la diferencia de las varianzas observada en las muestras originales
34 dif_obs <- dist_perm |> filter(id == "Apparent") |> pull(dif_v)
35
36 # Calcular el valor p
37 n_extremos <- dist_perm |> filter(abs(dif_v) >= abs(dif_obs)) |> summarise(n = n())
38 p_valor <- n_extremos |> pull() / (R + 1) # n_extremos ya incluye la diferencia observada
39
40 # Crear y mostrar situación de manera gráfica
41 titulo <- paste("Contraste de hipótesis usando", R, "permutaciones")
42 nota <- if_else(p_valor < 0.001, glue("italic(p)", " < ", "{sprintf('%.3f', 0.001)}"),
43               glue("italic(p)", " == ", "{sprintf('%.3f', p_valor)}"))
44 ancho_barra <- diff(range(dist_perm[["dif_v"]])) / nclass.scott(dist_perm[["dif_v"]])
45 ancho_barra_ajustado <- abs(dif_obs) / max(1, round(abs(dif_obs) / ancho_barra))
46
47 dist_perm_pl <- dist_perm |>
48   mutate(extremo = factor(if_else(abs(dif_v) >= abs(dif_obs), 1, 0))) |>
49   ggplot(aes(x = dif_v)) +
50   geom_histogram(aes(alpha = extremo, boundary = 0, binwidth = ancho_barra_ajustado,
51                     color = "steelblue4", fill = "steelblue")) +
52   labs(title = titulo, x = "Diferencia de dos varianzas", y = "Frecuencia") +
53   geom_vline(xintercept = c(-dif_obs, dif_obs), colour = "steelblue4", linetype = "dashed") +
54   annotate("text", x = dif_obs, y = Inf, hjust = 1.2, vjust = 2, label = nota, parse=TRUE) +
55   scale_alpha_manual(values = c("0" = 0.2, "1" = 1), guide = NULL) +
56   theme(plot.title = element_text(hjust = 0.5)) + theme_pubr()
57 print(dist_perm_pl)

```

El script 12.11 presenta la construcción de un gráfico con la situación del contraste de las hipótesis propuestas por el profesor. El código comienza rescatando el valor del estadístico de interés observado en el conjunto de datos originales (en la línea 34). En la siguiente sentencia (línea 37), se cuentan las remuestras que llevaron a estadísticos tan o más extremos que el observado. Como en este caso la distribución permutada ya incluye a este último valor, por lo que no tenemos que sumar 1 al numerador al calcular el valor p en la línea 38.

Luego, se define el título del gráfico y una etiqueta con el valor p obtenido (líneas 42–43), asegurando que se usen tres decimales y dando formato al texto por medio de la función `glue()`. A continuación (líneas 44–45) se calcula el ancho que debe tener cada una de las barras del histograma, el que se ajusta para que el valor observado coincida con el borde de una de ellas.

El script sigue con el *pipeline* para construir el histograma, que comienza añadiendo una columna, con nombre `extremo`, que indica si el valor del estadístico de esa remuestra es tanto o más extremo que el valor observado en las muestras originales (línea 48). Los siguientes dos pasos (líneas 49–50) generan el histograma para los valores de la columna `dif_v`, especificando el ancho de las barras y asegurando, para que este ancho funcione, que el valor cero se utilice como punto de referencia para calcular los límites de las barras (`boundary = 0`). Luego se agrega el título y las etiquetas de los ejes del gráfico (línea 52), se agregan líneas verticales en el

valor observado y su inverso para marcar las regiones críticas (línea 53) y la etiqueta que informa el valor p calculado. En siguiente paso (línea 55) se hace uso de la columna `extremo` para hacer más transparente las barras que no están en alguna de las dos áreas de la región crítica. El último paso del *pipeline* (línea 56) centra el título del gráfico y le aplica el estilo del paquete `ggpubr`.

La figura 12.13 muestra el gráfico que produce el script 12.11, donde vemos que la distribución permutada está centrada en el valor nulo (igual varianza) aunque presenta una leve asimetría negativa. La diferencia observada en las muestras originales ($\sigma_I^2 - \sigma_U^2 = 2,560$) tiene una baja probabilidad de ocurrir si la hipótesis nula fuera verdadera: $p^p = 0,002$. De esta forma, debemos rechazar esta hipótesis en favor de la alternativa y concluir que las varianzas de ambos grupos son distintas. Por el signo de la diferencia observada, la varianza de las calificaciones finales de estudiantes de primer año de Ingeniería parece ser significativamente mayor a la varianza de las calificaciones finales de estudiantes de último año de otras carreras en el curso estudiado.

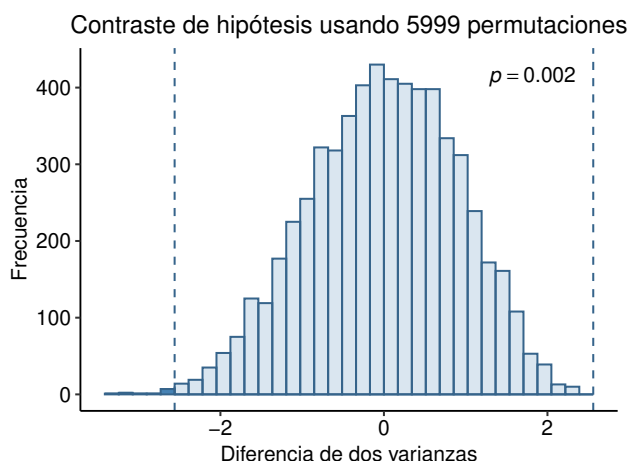


Figura 12.13: contraste de hipótesis usando una prueba de permutación Monte Carlo para la diferencia de dos varianzas.

Una observación final: los scripts 12.10 y 12.11 son bastante genéricos y no debería ser complicado modificarlos para aplicar pruebas de permutación a otros estadísticos de interés o para trabajar con **más de dos muestras independientes**.

12.1.7 Prueba de permutaciones para comparar más de dos muestras correlacionadas

Supongamos ahora que una estudiante de un curso de programación necesita comparar la eficiencia de tres algoritmos de ordenamiento: Quicksort, Bubblesort y Mergesort. Para ello, ha seleccionado aleatoriamente 6 arreglos de igual tamaño y registrado para cada uno de ellos el tiempo de ejecución utilizado por cada algoritmo (en milisegundos) bajo iguales condiciones, como muestra la tabla 12.3.

Instancia	Quicksort	Bubblesort	Mergesort
1	11,2	10,7	10,0
2	22,6	24,3	23,7
3	23,4	25,7	23,7
4	23,3	25,8	21,7
5	21,8	24,8	23,5
6	40,1	45,3	42,7

Tabla 12.3: tiempos de ejecución para las diferentes instancias con cada algoritmo del ejemplo.

Las hipótesis contrastadas son:

Sean t_i^Q , t_i^B y t_i^M los tiempos requeridos por los algoritmos de ordenamiento Quicksort, Bubblesort y Mergesort, respectivamente, para ordenar un arreglo i . Denotamos $(Y - X)$ al conjunto $\{t_i^Y - t_i^X\}$ de las diferencias en los tiempos de ejecución requeridos por los algoritmos X e Y en los mismos arreglos, y $\mu_{(Y - X)}$ a la media de estas diferencias. Entonces:

H_0 : no hay diferencias en el tiempo de ejecución medio necesitado por cada algoritmo de ordenamiento para ordenar las mismas instancias. Matemáticamente: $\mu_{(B - Q)} = \mu_{(M - Q)} = \mu_{(M - B)} = 0$.

H_A : la media de las diferencias en el tiempo de ejecución necesitado para ordenar las mismas instancias es diferente para al menos un par de algoritmos. Matemáticamente: $\exists X, Y \in \{Q, B, M\} \mid \mu_{(Y - X)} \neq 0$.

Por lo reducido de sus muestras, la estudiante ha decidido usar una prueba de permutación para resolver su problema. Para ello, ha considerado un nivel de significación $\alpha = 0,05$ y un total de 2.999 repeticiones. Con esta finalidad, la estudiante usó el script 12.12, que comienza definiendo las muestras de observaciones conseguidas en sus experimentos (líneas 12–15), para luego llevarlas a una matriz de datos en formato largo (líneas 17–20).

Script 12.12: prueba de permutación para la diferencia de tres medias correlacionadas.

```
1 library(dplyr)
2 library(forcats)
3 library(ggpubr)
4 library(glue)
5 library(purrr)
6 library(rsample)
7 library(rstatix)
8 library(tibble)
9 library(tidyr)
10
11 # Crear la matriz de datos y llevarla a formato largo
12 Quicksort <- c(11.2, 22.6, 23.4, 23.3, 21.8, 40.1)
13 Bubblesort <- c(10.7, 24.3, 25.7, 25.8, 24.8, 45.3)
14 Mergesort <- c(10.0, 23.7, 23.7, 21.7, 23.5, 42.7)
15 Instancia <- factor(paste0("I", 1:6))
16
17 datos anchos <- tibble(Instancia, Quicksort, Bubblesort, Mergesort)
18 datos_largos <- datos anchos |>
19   pivot_longer(~Instancia, names_to = "Algoritmo", values_to = "Tiempo") |>
20   mutate(Algoritmo = fct(Algoritmo))
21
22 # Reescribir la función que usa permutations() para crear las permutaciones
23 perm_splits_por_instancia <- function(data, times = 25) {
24   n <- nrow(data)
25   dl <- data |> select(Instancia) |> mutate(fila = row_number()) # línea agregada
26   # indices <- (rep(n, times), sample, replace = FALSE) # línea cambiada por:
27   indices <- map(1:times, ~ {dl |> slice_sample(by = Instancia, n = n) |> pull(fila)})
28   indices <- lapply(indices, rsample::perm_complement, n = n)
29   split_objs <- map(indices, make_splits, data = data, class = "perm_split")
30   list(splits = split_objs, id = rsample::names0(length(split_objs), "Permutations"))
31 }
32 assignInNamespace("perm_splits", perm_splits_por_instancia, ns = "rsample")
33
34 # Crear las remuestras
35 R <- 2999
36 set.seed(432)
37 remuestras <- permutations(datos_largos, permute = Tiempo, times = R, apparent = TRUE)
38
39 # Función que devuelve el estadístico F para las medidas repetidas en una remuestra
40 calc_F <- function(remuestra, ...) {
41   analysis(remuestra) |>
42     anova_test(dv = Tiempo, wid = Instancia, within = Algoritmo, type = 1) |> as_tibble() |>
43     pull(F)
44 }
45
46 # Obtener la distribución permutada de la diferencia de las medias
47 dist_perm <- remuestras |> mutate(F = map(splits, calc_F)) |> unnest(F)
```


Sin embargo, cuando intentó usar la función `permutations()` se percató de que esta no realiza permutaciones por bloque! Esto es problemático, pues para medidas repetidas no sería correcto intercambiar valores entre diferentes instancias, pues buscamos eliminar la variabilidad intracaso del análisis del efecto que tienen los algoritmos.

No tuvo más remedio que revisar el código fuente del paquete `rsample`, donde descubrió que `permutations()` hace uso de una función interna llamada `perm_splits()` donde existe una sentencia que crea las permutaciones de los índices que serán aplicados a la(s) columna(s) indicada(s). Hizo una copia de esta función en las líneas 23–31 del script 12.12, la que rebautizó como `perm_splits_por_instancia()`, y procedió a comentar la sentencia indeseada (línea 26). Para reemplazarla, primero creó una matriz auxiliar solo con los identificadores de las instancias en la matriz original y el índice de la fila de cada uno (línea 25), que luego utiliza para crear las permutaciones de los índices por instancia usando la función `slice_sample(data, n)` del paquete `dplyr` que selecciona `n` filas aleatoriamente desde la matriz de datos `data` (línea 27). Al especificar el argumento optativo `by = Instancia`, la función `slice_sample()` realiza la selección de filas con los datos agrupados por esta variable, es decir, dentro de cada instancia. De esta forma, al extraer la columna `fila` consigue los índices de las filas de la matriz de datos permutados por bloque. Luego, en la línea 32 del script, reemplaza la función original por la personalizada en el espacio de nombres del paquete `rsample`, para que las funciones de este paquete utilicen la versión modificada en lugar de la implementación original (durante la sesión actual de R).

Con la sobreescritura de esta función interna del paquete `rsample`, el resto del código es prácticamente el mismo que el visto en los scripts de la subsección anterior. La gran diferencia está en la función que calcula el estadístico de interés (líneas 40–44), que en este caso corresponde al estadístico F que usamos en el procedimiento ANOVA para muestras repetidas. Un detalle interesante: si bien la estudiante prefirió la elegancia de la filosofía *tidy* y utilizar la función `anova_test()` del paquete `rstatix`, hizo pruebas de rendimiento que indicaron que esta era más de 35 veces más lenta que usar la función base `aov()`. Investigando, se dio cuenta que por omisión la función utiliza suma de cuadrados de tipo 3. Recordando que cuando estudió sobre los procedimientos ANOVA leyó que para una vía los tipos de suma de cuadrados 1, 2 y 3 dan los mismos resultados, decidió probar con el parámetro `type = 1` que son más simples de calcular, y el tipo que usa la función `aov()`, con lo que logró aumentar la eficiencia de la función que ahora toma “solo” alrededor de 14 veces lo requerido por la función base.

El script 12.13 produce el gráfico de la situación encontrada al permutar el estadístico F, mostrada en la figura 12.14. Vemos que el estadístico observado tiene una baja probabilidad de ocurrir si la hipótesis nula omnibus es verdadera. Por esta razón, debemos rechazarla ($p^P = 0,009$) y concluir que al menos uno de los algoritmos requiere un tiempo de ejecución medio distinto al necesitado por al menos otro de los algoritmos.

Script 12.13: (continuación del script 12.12) gráfica del contraste de las hipótesis de igualdad de 3 medias correlacionadas usando una prueba de permutación.

```

49 # Obtener el estadístico F observado en las muestras originales
50 F_obs <- dist_perm |> filter(id == "Apparent") |> pull(F)
51
52 # Calcular el valor p
53 n_extremos <- dist_perm |> filter(F >= F_obs) |> summarise(n = n())
54 p_valor <- n_extremos |> pull() / (R + 1)
55
56 # Crear y mostrar la situación de manera gráfica
57 titulo <- sprintf("Contraste de hipótesis usando %d permutaciones", R)
58 nota <- if_else(p_valor < 0.001, glue("italic(p)", " < ", "{sprintf('%.3f', 0.001)}"),
59               glue("italic(p)", " == ", "{sprintf('%.3f', p_valor)}"))
60 dist_perm_pl <- dist_perm |>
61   mutate(extremo = factor(if_else(F >= F_obs, 1, 0))) |> ggplot(aes(x = F)) +
62   geom_histogram(aes(alpha = extremo), boundary = F_obs,
63                 binwidth = diff(range(dist_perm[["F"]])) / nclass.scott(dist_perm[["F"]]),
64                 color = "steelblue4", fill = "steelblue") +
65   labs(title = titulo, x = "F", y = "Frecuencia") +
66   geom_vline(xintercept = F_obs, colour = "steelblue4", linetype = "dashed") +
67   annotate("text", x = F_obs, y = Inf, hjust = 1.1, vjust = 2, label = nota, parse = TRUE) +
68   scale_alpha_manual(values = c("0" = 0.2, "1" = 1), guide = NULL) +
69   theme_pubr() + theme(plot.title = element_text(hjust = 0.5))
70 print(dist_perm_pl)

```

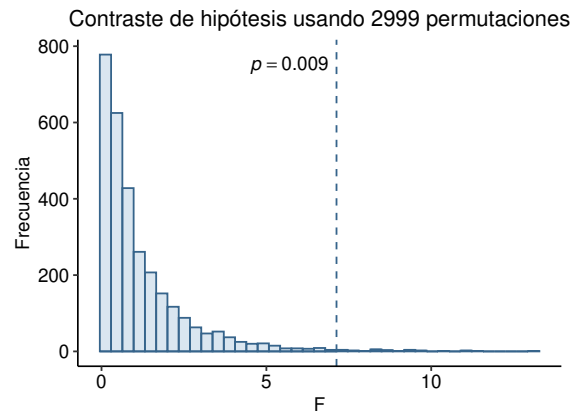


Figura 12.14: contraste de hipótesis usando una prueba de permutación Monte Carlo para la diferencia de más de dos medias correlacionadas.

Para proceder a realizar un análisis post-hoc, como es debido en estos casos, se deben utilizar las mismas permutaciones para obtener el nuevo estadístico de interés: las medias de las diferencias de los tiempos de ejecución registrados entre pares de algoritmos (*pairwise*). Para esto la estudiante agregó al programa en R el código que presenta el script 12.14.

Script 12.14: (continuación del script 12.13) análisis post-hoc para la prueba de permutación realizada.

```

72 # Función que devuelve las medias de las diferencias de los tiempos de ejecución
73 # entre pares de algoritmos según una remuestra.
74 medias_dif_pares <- function(remuestra, ...) {
75   dl <- analysis(remuestra)
76   dl |> get_comparisons(Algoritmo) |> unname() |>
77     map(~ {nombre <- paste(.x[1], .x[2], sep = " - ")
78         dl |>
79           reframe(difs = Tiempo[Algoritmo == .x[1]] - Tiempo[Algoritmo == .x[2]]) |>
80           summarise(!nombre := mean(difs))}
81     ) |> list_cbind()
82 }
83
84 # Obtener la distribución permutada de las medias de las diferencias de tiempo de
85 # ejecución entre pares de algoritmos.
86 posthoc_perm <- remuestras |> mutate(medias_dif = map(splits, medias_dif_pares)) |>
87   unnest(medias_dif)
88
89 # Obtener las medias de las diferencias de tiempo de ejecución entre pares de algoritmos
90 # observadas en las muestras originales.
91 posthoc_medias_obs <- posthoc_perm |> filter(id == "Apparent") |> select(-splits, -id)
92
93 # Calcular el valor p para cada caso
94 n_extremos <- posthoc_perm |>
95   summarise(across(.cols = where(is.numeric),
96     .fns = ~ sum(abs(.x) >= abs(posthoc_medias_obs[[cur_column()]])
97   ))
98 p_valores <- n_extremos / (R + 1)
99
100 # Producir una tabla con las medias de las diferencias entre cada par de algoritmos
101 # observadas en las muestras originales.
102 posthoc_tbl <- bind_rows(posthoc_medias_obs, p_valores, .id = "est") |>
103   mutate(est = c("Media dif. obs.", "p_perm")) |>
104   pivot_longer(-est, names_to = "Par de algoritmos", values_to = "Valor") |>
105   pivot_wider(names_from = est, values_from = Valor) |>
106   adjust_pvalue(p.col = "p_perm", output.col = "p_perm_adj", method = "BH") |>
107   mutate(across(where(is.numeric), ~ num(., digits = 3)))
108
109 # Mostrar los resultados del análisis post-hoc
110 msg <- "Resultados del análisis post-hoc"
111 cat(msg, ":\n", sep = "")
112 cat(rep("-", nchar(msg)), "\n", sep = "")
113 print(posthoc_tbl)

```

Primero define la función `medias_dif_pares()` para calcular el nuevo estadístico de interés para una remuestra (líneas 74–82). Con los datos permutados, obtiene los pares de algoritmos a comparar por medio de la función `get_comparisons()` del paquete `rstatix`; para cada par, construye un nombre para el resultado y calcula la media de las diferencias entre los algoritmos siendo comparados; finalmente, une los resultados en columnas de una matriz de datos de una fila utilizando la función `list_cbind()` del paquete `purrr`.

Esta función es utilizada en las líneas 86–87 para obtener la distribución permutada de las medias de diferencia entre pares de algoritmos. Debemos notar que desanida las matrices de datos devueltas por `medias_dif_pares()` para dejarlas como columnas simples para cada remuestra. Desde este matriz, recupera los valores calculados para los datos originales (línea 91). Luego, en las líneas 95–97, puede calcular los valores p correspondientes a cada par de algoritmos.

En las líneas 101–106, da formato a la información recolectada del análisis post-hoc, añadiendo una columna con los valores p ajustados con el método de Benjamini & Hochberg. Esta tabla es desplegada en pantalla en las líneas 109–112 del script, cuyo resultado puede verse en la figura 12.15.

```
Resultados del análisis post-hoc:
-----
# A tibble: 3 × 4
#   `Par de algoritmos`      `Media dif. obs.`      p_perm p_perm_adj
#   <chr>                  <num:.3!> <num:.3!> <num:.3!>
1 Quicksort - Bubblesort    -2.367      0.004      0.012
2 Quicksort - Mergesort     -0.483      0.622      0.622
3 Bubblesort - Mergesort     1.883      0.037      0.056
```

Figura 12.15: resultados del análisis post-hoc para la media de las diferencias entre pares de algoritmos.

Con estos resultados, la estudiante concluye, con 95 % confianza, que existen diferencias significativas en el tiempo promedio de ejecución entre los algoritmos Quicksort y Bubblesort ($p^p = 0,012$), y por el signo de la media de las diferencias observada entre ellos, Bubblesort es significativamente más lento que Quicksort.

12.2 MÉTODOS ROBUSTOS

Pensemos como ejemplo en la prueba t de Student (capítulo 5), que se usa para inferir acerca de la media de una población. Sin embargo, como mencionamos en el capítulo 2, esta medida de tendencia central tiene el problema de ser sensible a la presencia de valores atípicos, a distribuciones asimétricas o a muestras muy pequeñas. En términos generales, el incumplimiento de las condiciones del supuesto de normalidad puede causar diversos problemas:

- Resultados sesgados.
- Intervalos de confianza sub o sobreestimados.
- Reducción del poder estadístico de la prueba.

En el capítulo 2 mencionamos la existencia de estimadores robustos, poco sensibles a asimetrías muestrales o valores atípicos. No obstante, el paradigma estadístico tradicional no suele considerarlos. Esta sección, basada en las ideas expuestas por Mair y Wilcox (2020), aborda pruebas alternativas para muchas de las pruebas estudiadas hasta ahora, disponibles en el paquete [WRS2](#) de R, basadas en **estimadores robustos**.

12.2.1 Alternativas robustas a la media

En el capítulo 2 conocimos distintas medidas de tendencia central. Entre ellas vimos que la **mediana**, correspondiente al valor central (o el promedio de los dos valores centrales) de la muestra ordenada, es una alternativa robusta a la media. No obstante, existen otras opciones que nos pueden ser útiles.

La **media truncada** es bastante similar a la media aritmética que ya conocemos, con la diferencia de que se calcula **descartando** un determinado porcentaje (γ) de los valores en ambos extremos del conjunto de datos.

Tomemos como ejemplo una muestra X con 10 elementos, los cuales han sido ordenados por simplicidad:

$$X = \{5, 20, 37, 38, 40, 43, 43, 45, 87, 91\}$$

Si calculamos la media para la muestra anterior, tenemos que es $\bar{x} = 44,9$. No obstante, si observamos la muestra del ejemplo con detención, podemos darnos cuenta de que los valores extremos parecen ser atípicos y, en consecuencia, pueden tener una gran influencia en el valor resultante para la media. Así, podría ser más adecuado calcular la media truncada con $\gamma = 0,2$, es decir, podando el 20 % de los valores más pequeños y el 20 % de los valores más grandes, con lo que obtendremos:

$$\bar{x}^t = \frac{37 + 38 + 40 + 43 + 43 + 45}{6} = 41$$

En R, podemos calcular la media truncada mediante la ya conocida función `mean()` del paquete `base` (que, recordemos, se carga automáticamente al iniciar una sesión en R), agregando el argumento adicional `trim` con la proporción γ de los datos extremos a descartar, esto es `mean(x, trim = 0.2)` para el ejemplo. Notemos que si $\gamma = 0,5$ (`trim = 0.5`), se obtiene la mediana del conjunto de datos.

Un problema de la media truncada es que, al usarla, podríamos estar descartando muchos datos, lo que puede causar problemas cuando la muestra es pequeña. Otra opción puede ser, en lugar de descartar los valores extremos en cada cola, reemplazarlos por los valores extremos que no serían descartados al usar la media truncada y luego calcular la media con la muestra modificada. A esta medida se le conoce como **media Winsorizada**. Si retomamos nuestro ejemplo para la media truncada, los valores extremos tras la operación de truncado son 37 y 45. Así, reemplazamos los valores truncados en la muestra original por estos nuevos extremos, con lo que nuestra muestra Winsorizada es:

$$X^w = \{37, 37, 37, 38, 40, 43, 43, 45, 45, 45\},$$

y la media Winsorizada entonces sería:

$$\bar{x}^w = \frac{37 + 37 + 37 + 38 + 40 + 43 + 43 + 45 + 45 + 45}{10} = 41$$

En R, podemos hacer este cálculo mediante la función `winmean(x, tr)` del paquete `WRS2`, donde:

- `x`: vector con los datos originales.
- `tr`: proporción de los datos a Winsorizar en cada extremo.

Así, la llamada para el ejemplo sería `winmean(x, tr = 0.2)`.

12.2.2 Prueba de Yuen para dos muestras independientes

La **prueba de Yuen** es una buena alternativa a la prueba t de Student para muestras independientes que trabaja con las medias truncadas y Winsorizadas en vez de las medias aritméticas originales. De este modo, son una buena alternativa para comparar dos medias independientes cuando los datos no cumplen la condición de normalidad, presentan datos atípicos, las varianzas son muy diferentes o los tamaños de las muestras son muy dispares.

La prueba de Yuen para dos muestras independientes puede aplicarse si se cumplen las siguientes condiciones:

- Las observaciones en una muestra son independientes, esto significa que la elección de una observación no influye en la selección de otra para esa muestra.
- Las muestras son independientes, es decir que las observaciones de una muestra no están relacionadas con ninguna de las observaciones de la otra.
- La(s) variable(s) estudiada(s) tiene(n) al menos escala de intervalos iguales.

Sin embargo, hay otras condiciones que si bien no son obligatorias para aplicar el procedimiento, sí influyen en la calidad de las interpretaciones que podemos obtener:

- Las poblaciones de origen no son extremadamente diferentes (por ejemplo, una es muy sesgada a la derecha y la otra a la izquierda), por lo que comparar sus medias recortadas tiene sentido.
- El impacto de los valores extremos no es de interés de la investigación, pues la prueba de Yuen esencialmente ignora la información que estos valores entregan.
- El nivel de poda no está cerca del nivel de la mediana, siendo $\gamma \approx 0,2$ un valor frecuente.
- Las muestras no son demasiado reducidas, cuando la poda puede tener efectos perjudiciales. Como es usual, no existe un número fijo, pero algunos autores mencionan 5, 6 o 10 observaciones por cada muestra luego de la poda.

La prueba entonces se basa en la estimación de la **diferencia de las medias truncadas**: $d^t = \bar{x}_1^t - \bar{x}_2^t$, donde $\bar{x}_1^t$ y $\bar{x}_2^t$ son las medias truncadas de cada una de las muestras. El error estándar para este estadístico está dado por la ecuación 12.6:

$$SE_{d^t} = \sqrt{s_{X_1^t}^2 + s_{X_2^t}^2} = \sqrt{\frac{(n_1 - 1) s_{X_1^w}^2}{n_1^t (n_1^t - 1)} + \frac{(n_2 - 1) s_{X_2^w}^2}{n_2^t (n_2^t - 1)}} \quad (12.6)$$

siendo n_i el tamaño de la muestra i original, n_i^t el tamaño de la muestra i truncada, $s_{X_i^t}^2$ la varianza de la muestra i truncada y $s_{X_i^w}^2$ la varianza de la muestra i Winsorizada.

Así, el estadístico de prueba está dado por la ecuación 12.7:

$$T_y = \frac{d^t}{SE_{d^t}} = \frac{\bar{x}_1^t - \bar{x}_2^t}{\sqrt{s_{X_1^t}^2 + s_{X_2^t}^2}} \quad (12.7)$$

El estadístico T_y sigue una distribución t cuyos grados de libertad se calculan mediante la ecuación 12.8:

$$\nu_y = \frac{(s_{X_1^t}^2 + s_{X_2^t}^2)^2}{\frac{s_{X_1^t}^4}{n_1^t - 1} + \frac{s_{X_2^t}^4}{n_2^t - 1}} \quad (12.8)$$

Luego, es posible construir un intervalo de $100 \cdot (1 - \alpha) \%$ confianza como muestra la ecuación 12.9:

$$(\bar{x}_1^t - \bar{x}_2^t) \pm t^* \sqrt{s_{X_1^t}^2 + s_{X_2^t}^2}, \quad (12.9)$$

donde t^* corresponde al cuantil crítico $1 - \alpha/2$ de la distribución t con ν_y grados de libertad.

Como sugieren estas ecuaciones, las hipótesis contrastadas por la prueba de Yuen para dos muestras independientes son que las medias truncadas de las poblaciones de origen son iguales:

$$H_0: \mu_1^t = \mu_2^t$$

$$H_A: \mu_1^t \neq \mu_2^t$$

En R, podemos aplicar la prueba de Yuen para muestras independientes mediante una llamada a la función `yuen(formula, data, tr)` del paquete [WRS2](#), donde:

- **formula**: tiene la forma `<variable_dependiente> ~ <variable_independiente>`. Note que la variable independiente debe tener dos niveles, a fin de determinar a qué muestra pertenece cada observación de la variable dependiente.
- **data**: matriz de datos.
- **tr**: parámetro γ de la poda.

Veamos un ejemplo. El script 12.15 muestra el código que define dos muestras independientes del tiempo promedio de ejecución (en milisegundos) de dos algoritmos de complejidad computacional similar para resolver aproximadamente problemas de la un conjunto de 70 instancias del problema de ruteamiento de vehículos eléctricos con estaciones de carga intermedia, todas de igual tamaño y complejidad, que fueron construidas aleatoriamente. Estas instancias fueron asignadas al azar a cada uno de los algoritmos, resultando que $n_A = 40$ de ellas fueran resueltas por el algoritmo A y las restantes $n_B = 30$ por el algoritmo B.

Script 12.15: datos y graficación de dos muestras independientes.

```

1 library(ggpubr)
2 library(WRS2)
3
4 # Construir la matriz de datos
5 a <- c(25.1, 25.2, 25.3, 25.3, 25.4, 25.4, 25.5, 25.5, 25.6, 25.8, 25.8, 25.9, 25.9,
6       26.0, 26.0, 26.2, 26.2, 26.2, 26.3, 26.4, 26.5, 26.5, 26.6, 26.7, 26.7, 26.9,
7       26.9, 27.0, 27.1, 27.3, 27.8, 28.4, 28.5, 29.0, 29.8, 30.2, 31.8, 31.9, 33.3, 33.7)
8
9 b <- c(24.1, 24.4, 24.4, 24.5, 24.7, 24.8, 24.8, 25.1, 25.2, 25.2, 25.2, 25.3, 25.4, 25.7,
10      25.7, 26.3, 26.3, 26.4, 26.5, 27.2, 27.7, 28.3, 28.4, 28.4, 28.6, 28.7, 29.6, 29.9,
11      30.1, 30.5)
12
13 Tiempo <- c(a, b)
14 Algoritmo <- c(rep("A", length(a)), rep("B", length(b)))
15 datos <- data.frame(Tiempo, Algoritmo)
16
17 # Comprobar normalidad
18 qq <- ggqqplot(datos, x = "Tiempo", facet.by = "Algoritmo",
19               color = "Algoritmo", palette = c("steelblue", "steelblue1"),
20               xlab = "Cuantil teórico", ylab = "Tiempo de ejecución [ms]")
21 qq <- qq + theme(legend.position = "none")
22 print(qq)

```

Las líneas 19–22 del script 12.15 construyen gráficos Q-Q para comprobar el supuesto de normalidad requerido por la prueba t de Student para dos muestras independientes, obteniéndose como resultado la figura 12.16, donde podemos observar que las muestras obtenidas presentan desviaciones importantes de una distribución normal, especialmente para el caso del algoritmo A. Por esta razón, **no podemos** utilizar una prueba t de Student para inferir sobre la diferencia del rendimiento medio de los algoritmos A y B para determinar si tienen igual desempeño o alguno de ellos es más eficiente.

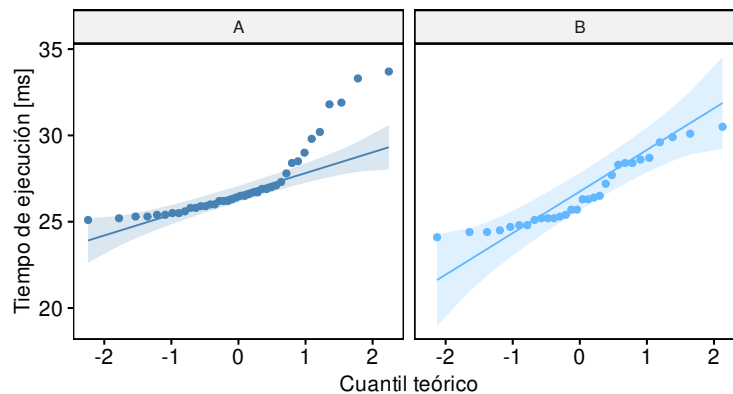


Figura 12.16: gráfico Q-Q de las muestras independientes originales.

Como alternativa para analizar estos datos usaremos la prueba de Yuen. Por la descripción del experimento, podemos confiar en que las condiciones de independencia al interior y entre las muestras se cumple, al igual que una escala adecuada. Por otro lado, como ambos algoritmos tienen complejidades computacionales parecidas, podríamos esperar que las distribuciones de sus tiempos de ejecución no sean muy diferentes. Además, las muestras son lo suficientemente grandes como para poder aplicar una poda razonable. Si usamos $\gamma = 0.2$, las muestras truncadas tendrían, respectivamente, $n_1^t = 24$ y $n_2^t = 24$ observaciones.

Para ilustrar los conceptos asociados a la prueba de Yuen, el script 12.16 trunca ambas muestras considerando $\gamma = 0.2$ y construyen gráficos Q-Q para las muestras podadas, los que pueden verse en la figura 12.17. Podemos apreciar que, tras la poda, los datos siguen distribuciones que se aproxima más a la normal.

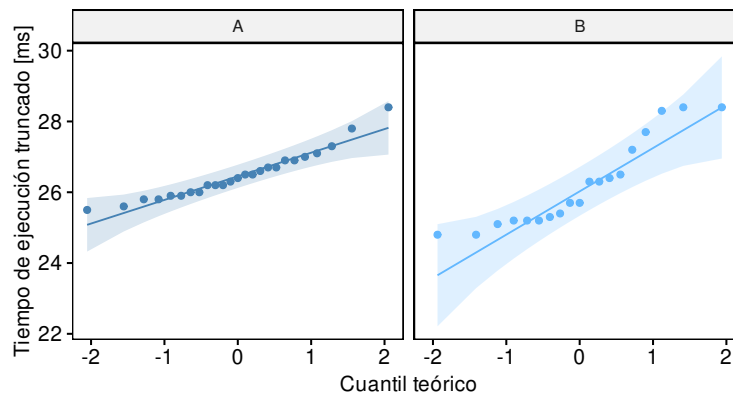


Figura 12.17: gráfico Q-Q de las muestras independientes truncadas.

Por supuesto, la aplicación del procedimiento es más simple utilizando la función `yuen()`, como muestra la línea 45 del script 12.17.

Script 12.16: (continuación del script 12.15) poda y graficación de las dos muestras independientes truncadas.

```

24 # Aplicar una poda del 20% a las muestras
25 gamma <- 0.2
26 n_a <- length(a)
27 n_b <- length(b)
28 poda_a <- floor(n_a * gamma)
29 poda_b <- floor(n_b * gamma)
30
31 a_trunc <- a[poda_a:(n_a - poda_a)]
32 b_trunc <- b[poda_b:(n_b - poda_b)]
33
34 Tiempo_t <- c(a_trunc, b_trunc)
35 Algoritmo_t <- c(rep("A", length(a_trunc)), rep("B", length(b_trunc)))
36 datos_t <- data.frame(Tiempo_t, Algoritmo_t)
37
38 qq_t <- ggqqplot(datos_t, x = "Tiempo_t", facet.by = "Algoritmo_t",
39                 color = "Algoritmo_t", palette = c("steelblue", "steelblue1"),
40                 xlab = "Cuantil teórico", ylab = "Tiempo de ejecución truncado [ms]")
41 qq_t <- qq_t + theme(legend.position = "none")
42 print(qq_t)

```

La figura 12.18 muestra el resultado de ejecutar este script. Observamos que obtenemos una diferencia entre las medias truncadas de 0,246 [ms], con intervalo de 95 % confianza $[-0,859; 1,351]$ [ms] y tamaño del efecto de 0,100. La prueba no resulta significativa ($T_y(29,05) = 0,455$; $p = 0,653$; $\gamma = 0,2$) al nivel de significación $\alpha = 0,05$, por lo que concluimos con 95 % confianza que no es posible descartar que ambos algoritmos tienen, en promedio, igual tiempo de ejecución.

El paquete `WRS2` incluye también la función `pb2gen(formula, data, est, nboot)`, que usa bootstrapping para aplicar la prueba de Yuen usando otras medidas robustas de tendencia central, donde:

- **formula**: tiene la misma forma descrita para la prueba de Yuen.
- **data**: matriz de datos.
- **est**: medida a emplear. Puede tomar las opciones `"mean"` para la media y `"median"` para la mediana, entre otras opciones que escapan a los alcances de este curso.
- **nboot**: cantidad de repeticiones bootstrap.

El script 12.17 muestra cómo usar la función `pb2gen()` en el ejemplo, usando como estimadores la media (línea 53) y la mediana (línea 57), obteniéndose los resultados de la figura 12.19. Podemos ver que, consistentemente en ambos casos, las pruebas tampoco resultan significativas ($T_y \leq 0,610$; $p \geq 0,213$; $\gamma = 0,2$; 999 iteraciones bootstrap).


```

Prueba de Yuen para dos muestras independientes
-----
Call:
yuen(formula = Tiempo ~ Algoritmo, data = datos, tr = gamma)

Test statistic: 0.455 (df = 29.05), p-value = 0.65252

Trimmed mean difference: 0.24583
95 percent confidence interval:
-0.8592      1.3509

Explanatory measure of effect size: 0.1

```

Figura 12.18: resultado de la prueba de Yuen para las muestras independientes del ejemplo.

Script 12.17: (continuación del script 12.16) prueba de Yuen para dos muestras independientes asintótica y usando bootstrapping.

```

44 # Aplicar y mostrar la prueba de Yuen asintótica
45 prueba <- yuen(Tiempo ~ Algoritmo, data = datos, tr = gamma)
46 cat("\nPrueba de Yuen para dos muestras independientes\n")
47 cat("-----\n")
48 print(prueba)
49
50 # Aplicar la prueba de Yuen con bootstrapping y la media
51 R <- 999
52 set.seed(135)
53 prueba_media <- pb2gen(Tiempo ~ Algoritmo, data = datos, est = "mean", nboot = R)
54
55 # Aplicar la prueba de Yuen con bootstrapping y la mediana
56 set.seed(135)
57 prueba_mediana <- pb2gen(Tiempo ~ Algoritmo, data = datos, est = "median", nboot = R)
58
59 # Mostrar los resultados
60 cat("\nPrueba de Yuen - implemetación con bootstrapping\n")
61 cat("=====\n")
62 cat("\nResultado al usar bootstrapping y la media como estimador\n")
63 cat("-----\n")
64 print(prueba_media, )
65 cat("\nResultado al usar bootstrapping y la mediana como estimador\n")
66 cat("-----\n")
67 print(prueba_mediana)

```

12.2.3 Prueba de Yuen para dos muestras pareadas

La prueba de Yuen, al igual que la prueba t de Student, fue adaptada para trabajar con dos muestras pareadas y es una alternativa cuando los datos no cumplen la condición de normalidad o existen valores atípicos. Las suposiciones de esta prueba son:

- Los pares de observaciones son independientes, es decir que la elección de un par no influye en la selección de otro.
- La variable medida tiene al menos escala de intervalos iguales.

Como cuando se trabaja con muestras independientes, hay otras condiciones que afectan la calidad de esta versión de la prueba:

- Las diferencias siguen una distribución relativamente simétrica.
- La poda que se aplica elimina valores atípicos extremos.
- El nivel de poda no es cercano al nivel de la mediana.
- Las muestras no son demasiado reducidas. Algunos autores mencionan que se deberían tener al menos 10 o 15 pares de observaciones.

```

Prueba de Yuen - implementación con bootstrapping
=====

Resultado al usar bootstrapping y la media como estimador
-----
Call:
pb2gen(formula = Tiempo ~ Algoritmo, data = datos, est = "mean",
        nboot = R)

Test statistic: 0.61, p-value = 0.21321
95% confidence interval:
-0.3008      1.5617

Resultado al usar bootstrapping y la mediana como estimador
-----
Call:
pb2gen(formula = Tiempo ~ Algoritmo, data = datos, est = "median",
        nboot = R)

Test statistic: 0.45, p-value = 0.47147
95% confidence interval:
-0.95      1.35

```

Figura 12.19: resultado de la prueba de Yuen con bootstrapping para las muestras independientes del ejemplo, usando como estimadores la media y la mediana.

Para el caso de dos muestras apareadas, como ha sido usual, debemos considerar el conjunto de las diferencias entre pares de observaciones:

$$D = \{d_i = x_i^{(1)} - x_i^{(2)}\},$$

siendo $(x_i^{(1)}, x_i^{(2)})$ el i -ésimo par de observaciones en la muestra. Luego podemos aplicar la poda a este conjunto y calcular la media truncada de las diferencias: $\bar{D}^t$.

El error estándar de esta media truncada está dada por la ecuación 12.10:

$$SE_{\bar{D}^t} = \sqrt{\frac{(n-1)s_{D^w}^2}{n^t(n^t-1)}} \quad (12.10)$$

donde n es el número de pares en la muestra original D , n^t es la cantidad de pares disponibles luego de la poda, y s_{D^w} es la desviación estándar del conjunto de las diferencias Winsorizado.

Así, el estadístico de prueba para datos pareados sería como muestra la ecuación 12.11:

$$T_y = \frac{\bar{D}^t}{\sqrt{\frac{(n-1)s_{D^w}^2}{n^t(n^t-1)}}}, \quad (12.11)$$

que sigue una distribución t con $n^t - 1$ grados de libertad, lo que permite obtener valores p e intervalos de confianza.

Nuevamente estas ecuaciones sugieren que esta extensión de la prueba de Yuen contrasta hipótesis bilaterales, que en este caso corresponde a que la media truncada de las diferencias apareadas es cero:

$$H_0: \mu_D^t = 0$$

$$H_A: \mu_D^t \neq 0$$

En R, podemos aplicar la prueba de Yuen para muestras apareadas mediante una llamada a la función `yuend(x, y, tr)` del paquete `WRS2`, donde:

- `x`: vector numérico con la primera muestra.
- `y`: vector numérico con la segunda muestra.
- `tr`: parámetro γ de la poda.

Como es de esperar, la función falla si los largos de los vectores $\bar{x}$ e $\bar{y}$ no coinciden.

Consideremos un ejemplo. Supongamos ahora que queremos comparar el rendimiento de los algoritmos A y B para el problema de ruteamiento de vehículos eléctricos con estaciones de carga intermedia, descritos en la sección anterior, para lo cual esta vez hemos seleccionado aleatoriamente 25 instancias del problema y registrado el tiempo de ejecución en milisegundos que cada uno tarda en resolverlas.

El script 12.18 muestra los tiempos de ejecución observados para cada algoritmo (líneas 5–8). El resto del script construye gráficos Q-Q para el conjunto de las diferencias entre los pares de observaciones y para este conjunto podado (obtenido en las líneas 12–16) aplicando un recorte $\gamma = 0,2$.

Script 12.18: datos y graficación de las diferencias en muestras apareadas.

```
1 library(ggpubr)
2 library(WRS2)
3
4 # Construir las estructuras con los datos observados
5 a <- c(32.3, 32.0, 32.0, 36.0, 34.2, 32.7, 32.5, 32.0, 32.1, 33.4, 32.3, 37.2, 32.1,
6       32.0, 33.9, 34.1, 36.6, 34.5, 32.7, 33.1, 32.7, 32.1, 36.7, 32.2, 38.0)
7 b <- c(35.3, 20.1, 18.6, 46.3, 42.1, 39.3, 37.0, 28.0, 30.2, 40.4, 35.6, 50.7, 33.6,
8       17.9, 41.0, 41.6, 47.8, 43.2, 38.3, 39.9, 38.0, 28.3, 48.4, 34.7, 52.9)
9 dif <- a - b
10
11 # Aplicar una poda del 20% al conjunto de diferencias y juntar los datos en una matriz
12 gamma <- 0.2
13 n <- length(dif)
14 poda <- floor(n * gamma)
15 dif <- sort(dif)
16 dif_trunc <- dif[(poda + 1):(n - poda)]
17 n_t <- length(dif_trunc)
18
19 datos <- data.frame(Diferencia = c(dif, dif_trunc),
20                    Muestra = c(rep("Original", n), rep("Podados", n_t)))
21
22 # Obtener gráficos Q-Q de las diferencias originales y podadas
23 qq <- ggqqplot(datos, x = "Diferencia", facet.by = "Muestra",
24               palette = c("steelblue", "steelblue1"), color = "Muestra",
25               xlab = "Cuantil teórico",
26               ylab = "Diferencias en tiempos de ejecución\nen las mismas instancias [ms]")
27 qq <- qq + theme(legend.position = "none")
28 print(qq)
```

Los gráficos resultantes se muestran en la figura 12.20, donde podemos ver que el conjunto de diferencias observadas (a la izquierda) exhibe desviaciones importantes de normalidad, por lo que no sería prudente asumir que esta proviene de una distribución normal. En cambio, el conjunto podado de estas observaciones (a la derecha) muestra un comportamiento más esperado para una muestra que proviene de una población normal. Así, es conveniente que usemos la prueba de Yuen para dos muestras pareadas en este caso, como alternativa a la prueba t pareada.

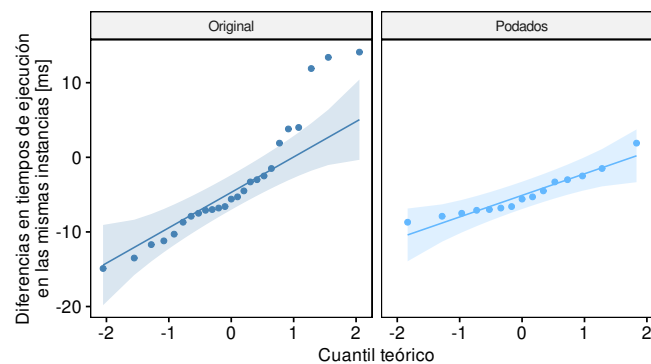


Figura 12.20: gráficos Q-Q de las diferencias originales y truncadas.

El enunciado del ejemplo sugiere que las condiciones de independencia entre los pares de observaciones se estaría cumpliendo. Por otro lado, los gráficos Q-Q de las diferencias en tiempos de ejecución (figura 12.20) muestran que existe bastante asimetría hacia la cola inferior en la muestra original, aunque se resuelve bien al considerar la muestra truncada, las que no incluyen valores atípicos extremos.

Si bien el nivel de poda usado en este gráfico es razonable, el tamaño de la muestra es bastante reducida, con solo 15 pares de observaciones. Por esta última razón y la asimetría que podría existir en la población de origen, seremos prudentes y usaremos un nivel de significación exigente: $\alpha = 0,01$.

Entonces, para el ejemplo vamos a contrastar las siguientes hipótesis:

H_0 : Sin considerar casos extremos, en promedio, los algoritmos A y B tardan lo mismo en resolver las mismas instancias de prueba. Matemáticamente: si D es la distribución de las diferencias en el tiempo de ejecución que tardan los algoritmos en resolver las mismas instancias, entonces la media truncada de estas diferencias es nula: $\mu_D^t = 0$.

H_A : A pesar de no considerar los casos extremos, los algoritmos A y B , en promedio, no toman el mismo tiempo de ejecución para resolver las mismas instancias de prueba. Es decir, $\mu_D^t \neq 0$

El script 12.19 ilustra el uso de la función `yuend()` para el ejemplo, obteniéndose el resultado que muestra la figura 12.21.

Script 12.19: (continuación del script 12.18) prueba de Yuen para dos muestras apareadas.

```
30 # Aplicar y mostrar la prueba de Yuen para muestras apareadas
31 prueba <- yuend(x = a, y = b, tr = gamma)
32 cat("Prueba de Yuen para dos muestras pareadas\n")
33 cat("-----\n")
34 print(prueba)
```

Podemos ver que la prueba resulta significativa ($T_y(14) = -3,584$; $p = 0,003$; $\gamma = 0,2$) para el nivel de significación establecido. Así, debemos concluir con 99 % confianza que existe una diferencia estadísticamente significativa en el desempeño de ambos algoritmos, siendo el algoritmo A el más eficiente (la función `yuend()` solo nos permite reportar el intervalo de 95 % confianza: $[-8,035; -2,019]$).

```
Prueba de Yuen para dos muestras pareadas
-----
Call:
yuend(x = a, y = b, tr = gamma)

Test statistic: -3.5843 (df = 14), p-value = 0.00299

Trimmed mean difference:  -5.02667
95 percent confidence interval:
-8.0346      -2.0188

Explanatory measure of effect size: 0.72
```

Figura 12.21: resultado de la prueba de Yuen para las muestras apareadas del ejemplo.

12.2.4 Análisis robusto de una vía para muestras independientes

El paquete `WRS2` ofrece diferentes alternativas a ANOVA de una vía para muestras independientes que podemos usar cuando los tamaños muestrales son muy diferentes o no se cumple la condición de homocedasticidad. Todas ellas asumen que:

- Las observaciones en una muestra son independientes, es decir que las observación se eligen sin considerar ninguna otra.
- Las muestras son independientes, es decir que las observaciones de una muestra no tienen ninguna relación con alguna de las observaciones en las otras muestras.
- La(s) variable(s) estudiada(s) tiene(n) al menos escala de intervalos iguales.

Sin embargo, también debemos tener en cuenta condiciones que afectan la calidad de la prueba, que son análogas a las mencionadas para la prueba de Yuen.

La función `t1way(formula, data, tr, alpha)` efectúa un procedimiento similar a ANOVA usando medias truncadas. A su vez, la función `lincon(formula, data, tr, alpha)` permite realizar el procedimiento post-hoc correspondiente. De manera similar, `t1waybt(formula, data, tr, nboot)` realiza un procedimiento análogo al anterior incorporando bootstrapping. En este caso, el procedimiento post-hoc puede realizarse mediante la función `mcpb20(formula, data, tr, nboot)`.

Una tercera opción es la función `mediway(formula, data, iter)`, que emplea la mediana y sigue un proceso iterativo. No obstante, en este caso el paquete no ofrece funciones que permitan realizar el procedimiento post-hoc. Los argumentos asociados a las funciones mencionadas en los párrafos anteriores son:

- **formula**: de la forma `<variable_dependiente> ~ <variable_independiente>`.
- **data**: matriz de datos.
- **tr**: parámetro γ de la poda.
- **alpha**: nivel de significación.
- **nboot**: cantidad de repeticiones bootstrapping.
- **iter**: cantidad de iteraciones a realizar.

Para ejemplificar pensemos que ahora deseamos comparar el tiempo promedio de ejecución (en milisegundos) de tres algoritmos que resuelven instancias distintas del problema de ruteamiento de vehículos eléctricos con estaciones de carga intermedia, contando con $n_A = 40$ observaciones para el algoritmo *A*, $n_B = 30$ observaciones para el *B* y $n_C = 35$ observaciones para el *C*. El script 12.20 presenta estas observaciones (líneas 5–15). Considerando que esta comparación podría realizarse por medio de un procedimiento ANOVA para muestras independientes, el script también produce gráficos Q-Q para las muestras (líneas 23–27), los que son presentados en la figura 12.22, donde se observa que las observaciones recogidas presentan desviaciones importantes del comportamiento esperado para datos que provienen de distribuciones normales, por lo que no podríamos utilizar el procedimiento ANOVA, al menos de manera directa.

Sin embargo, la figura 12.22 también sugiere que las desviaciones ocurren en los extremos de las muestras, por lo que razonablemente podríamos suponer que comparar medias truncadas podría ser más adecuado. Como la variable dependiente es una medición física (tiempo), ya sabemos que cumple con la escala de intervalos iguales. Además la descripción del ejemplo sugiere que las muestras también cumplen las condiciones de independencia en cada muestra y entre ellas.

De esta forma, podemos aplicar el análisis robusto de una vía para muestras independientes, considerando un nivel de significación $\alpha = 0,05$, con las siguientes hipótesis:

H_0 : el tiempo de ejecución promedio necesitado para resolver instancias de igual tamaño es la misma para los tres algoritmos. Matemáticamente: $\mu_A = \mu_B = \mu_C$.

H_A : el tiempo de ejecución promedio necesitado para resolver instancias de igual tamaño es diferente para al menos un algoritmo. Matemáticamente: $\exists X, Y \in \{A, B, C\} \mid \mu_X \neq \mu_Y$.

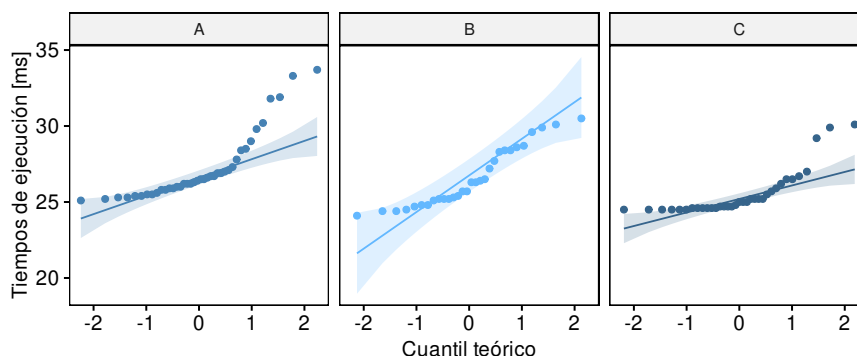


Figura 12.22: gráficos Q-Q de las observaciones del ejemplo.

El script 12.20 a continuación ilustra el funcionamiento de las funciones `t1way()` (línea 36) y `lincon()` (línea 43) del paquete `WRS2` que aplican, respectivamente, las pruebas robustas ómnibus y post-hoc para el análisis de las muestras independientes del ejemplo. Quien preste atención ha de notar que antes de la llamada a la prueba ómnibus hemos fijado una semilla para la reproducibilidad de números aleatorios (línea 35) y que, en la llamada, hemos indicado el argumento `nboot = R`. Debemos aclarar que esta prueba es **asintótica**, como la gran mayoría de las pruebas que hemos estudiado, pero los procedimientos para estimar el **tamaño del efecto** que reporta y su **intervalo de confianza** sí hacen uso de **bootstrapping**. Esta es la razón por la que hemos definido una semilla para números aleatorios fija y un número de iteraciones bootstrap. También debemos observar que no hemos indicado un método para ajustar los valores p de las comparaciones post-hoc, por lo que la función `lincon()` por omisión utiliza el procedimiento de Benjamini y Hochberg.

Script 12.20: análisis robusto de una vía para comparar tres muestras independientes.

```

1 library(ggpubr)
2 library(WRS2)
3
4 # Construir las estructuras con los datos
5 A <- c(25.1, 25.2, 25.3, 25.3, 25.4, 25.4, 25.5, 25.5, 25.6, 25.8, 25.8, 25.9, 25.9, 26.0,
6       26.0, 26.2, 26.2, 26.2, 26.3, 26.4, 26.5, 26.5, 26.6, 26.7, 26.7, 26.9, 26.9, 27.0,
7       27.1, 27.3, 27.8, 28.4, 28.5, 29.0, 29.8, 30.2, 31.8, 31.9, 33.3, 33.7)
8
9 B <- c(24.1, 24.4, 24.4, 24.5, 24.7, 24.8, 24.8, 25.1, 25.2, 25.2, 25.2, 25.3, 25.4, 25.7,
10      25.7, 26.3, 26.3, 26.4, 26.5, 27.2, 27.7, 28.3, 28.4, 28.4, 28.6, 28.7, 29.6, 29.9,
11      30.1, 30.5)
12
13 C <- c(24.5, 24.5, 24.5, 24.5, 24.5, 24.5, 24.6, 24.6, 24.6, 24.6, 24.6, 24.6, 24.7, 24.7,
14      24.7, 24.7, 24.8, 25.0, 25.0, 25.0, 25.2, 25.2, 25.2, 25.2, 25.5, 25.7, 25.9, 26.2,
15      26.5, 26.5, 26.7, 27.0, 29.2, 29.9, 30.1)
16
17 Tiempo <- c(A, B, C)
18 Algoritmo <- c(rep("A", length(A)), rep("B", length(B)), rep("C", length(C)))
19 Algoritmo <- factor(Algoritmo)
20 datos <- data.frame(Tiempo, Algoritmo)
21
22 # Obtener gráficos Q-Q de las muestras
23 qq <- ggqqplot(datos, x = "Tiempo", facet.by = "Algoritmo",
24              color = "Algoritmo", palette = c("steelblue", "steelblue1", "steelblue4"),
25              xlab = "Cuantil teórico", ylab = "Tiempos de ejecución [ms]")
26 qq <- qq + theme(legend.position = "none")
27 print(qq)
28
29 # Fijar nivel de significación, el nivel de poda y el número de repeticiones
30 alfa <- 0.05
31 gamma <- 0.2
32 R <- 999
33
34 # Comparar los diferentes algoritmos usando medias truncadas
35 set.seed(666)
36 una_via <- t1way(Tiempo ~ Algoritmo, data = datos, tr = gamma, alpha = alfa, nboot = R)
37
38 cat("Análisis de una vía para muestras independientes (asintótico)\n")
39 cat("-----\n")
40 print(una_via)
41
42 if(una_via[["p.value"]] < alfa) {
43   una_via_ph <- lincon(Tiempo ~ Algoritmo, data = datos, tr = gamma, alpha = alfa)
44
45   cat("Análisis post-hoc para muestras independientes (asintótico)\n")
46   cat("-----\n")
47   print(una_via_ph)

```

El resultado de ejecutar este trozo de código puede verse en la figura 12.23. Vemos que la prueba robusta ómnibus resulta significativa ($F(2; 34,39) = 10,981$; $p < 0,001$; $\gamma = 0,2$) al nivel establecido, por lo que debemos rechazar la hipótesis nula. Concluimos, entonces, con 95 % confianza, que existe una diferencia estadísticamente significativa entre los tiempos promedio de ejecución de los algoritmos cuando no se consideran casos extremos.

```

Análisis de una vía para muestras independientes (asintótico)
-----
Call:
t1way(formula = Tiempo ~ Algoritmo, data = datos, tr = gamma,
      alpha = alfa, nboot = R)

Test statistic: F = 10.9813
Degrees of freedom 1: 2
Degrees of freedom 2: 34.39
p-value: 0.00021

Explanatory measure of effect size: 0.46
Bootstrap CI: [0.27; 0.67]

Análisis post-hoc para muestras independientes (asintótico)
-----
Call:
lincon(formula = Tiempo ~ Algoritmo, data = datos, tr = gamma,
      alpha = alfa)

      psihat ci.lower ci.upper p.value
A vs. B 0.24583 -1.11867  1.61033 0.65252
A vs. C 1.49583  0.65571  2.33596 0.00022
B vs. C 1.25000 -0.02757  2.52757 0.03909

```

Figura 12.23: resultado de aplicar pruebas robustas ómnibus y post-hoc para comparar más de dos muestras independientes con un factor (una vía).

El procedimiento post-hoc reporta una tabla con las estimaciones de `psihat` que denota las diferencias de las medias truncadas entre cada par de algoritmos, acompañadas de su intervalo de confianza y el correspondiente valor p ajustado. Observando la figura 12.23, podemos concluir que, sin considerar los casos extremos, el algoritmo C presenta un tiempo de ejecución promedio más bajo que los algoritmos A ($\bar{x}_A^t - \bar{x}_C^t = 1,496$ [ms]; 95 % CI*: [0,656; 2,336] [ms]; $p < 0,001$; $\gamma = 0,2$) y B ($\bar{x}_B^t - \bar{x}_C^t = 1,250$ [ms]; 95 % CI*: [0,028; 2,028] [ms]; $p = 0,039$; $\gamma = 0,2$).

Por otro lado, el script 12.21 muestra el uso de las funciones `t1waybt()` y `mcpb20()` que aplican bootstrapping para el análisis robusto ómnibus y post-hoc a las muestras independientes del ejemplo. Nuevamente debemos hacer una observación: no hemos definido un nivel de significación para el cálculo de intervalos de confianza. Esto se debe a que la función para aplicar el procedimiento ómnibus no estima tal intervalo, como hace la función `t1way()`, mientras que la implementación del procedimiento post-hoc ¡siempre utiliza $\alpha = 0,05$!

Además, cabe destacar que la función `mcpb20()` ajusta los múltiples valores p utilizando una aproximación basada en las distribuciones bootstrap generadas de manera de controlar rigurosamente la tasa de error por familia.

Script 12.21: (continuación del script 12.20) análisis robusto de una vía para muestras independientes aplicando bootstrapping.

```

49
50 # Comparar los diferentes algoritmos usando medias truncadas y bootstrapping
51 set.seed(666)
52 una_via_bt <- t1waybt(Tiempo ~ Algoritmo, data = datos, tr = gamma, nboot = R)
53
54 cat("Análisis de una vía para muestras independientes (bootstrapped)\n")
55 cat("-----\n")
56 print(una_via_bt)
57
58 if(una_via_bt[["p.value"]] < alfa) {
59   set.seed(666)
60   una_via_bt_ph <- mcpb20(Tiempo ~ Algoritmo, data = datos, tr = gamma, nboot = R)
61
62   cat("Análisis post-hoc para muestras independientes (bootstrapped)\n")
63   cat("-----\n")
64   print(una_via_bt_ph)
65 }

```



```

Análisis de una vía para muestras independientes (bootstrapped)
-----
Call:
tiwaybt(formula = Tiempo ~ Algoritmo, data = datos, tr = gamma,
        nboot = R)

Effective number of bootstrap samples was 999.

Test statistic: 10.9813
p-value: 0.001
Variance explained: 0.179
Effect size: 0.424

Análisis post-hoc para muestras independientes (bootstrapped)
-----
Call:
mcppb20(formula = Tiempo ~ Algoritmo, data = datos, tr = gamma,
        nboot = R)

      psihat ci.lower ci.upper p-value
A vs. B 0.24583 -0.91667  1.61389 0.55656
A vs. C 1.49583  0.73690  2.44464 0.00000
B vs. C 1.25000  0.16270  2.34206 0.00801

```

Figura 12.24: resultados al aplicar pruebas ómnibus y post-hoc robustas que utilizan bootstrapping en el análisis de una vía a las muestras independientes del ejemplo.

El resultado que entrega el script 12.21 es mostrado en en la figura 12.24, donde vemos que la prueba robusta ómnibus resulta significativa, reportando el mismo estadístico que el procedimiento asintótico, con un valor p estimado vía bootstrapping algo mayor ($p^* = 0,001$), confirmando la existencia de diferencias estadísticamente significativas en los tiempos de ejecución requeridos por los algoritmos cuando no se consideran casos extremos. El procedimiento post-hoc con bootstrapping también confirma que, con 95 % confianza, sin considerar los casos extremos, el algoritmo *C* presenta un tiempo de ejecución promedio más bajo que los algoritmos *A* (95 % CI*: [0,734; 2,444] [ms]; $p^* < 0,001$; $\gamma = 0,2$; $R = 999$) y *B* (95 % CI*: [0,163; 2,342] [ms]; $p^* = 0,008$; $\gamma = 0,2$; $R = 999$).

12.2.5 Análisis robusto de una vía para muestras correlacionadas

Desde luego, el paquete [WRS2](#) también ofrece opciones robustas para reemplazar el procedimiento ANOVA de una vía para muestras correlacionadas, que podemos usar cuando los datos disponibles violan la condición de normalidad o de esfericidad. Estos procedimientos robustos asumen las siguientes condiciones:

- Los casos o bloques medidos son independientes entre sí.
- Se tiene un conjunto de mediciones (usualmente mayor a dos) para cada caso o bloque.
- Las variable medida tiene al menos escala de intervalos iguales.

También debemos tener en cuenta las condiciones que afectan la calidad de la prueba, que son análogas a las mencionadas para la prueba de Yuen con muestras apareadas.

La función `rmanova(y, groups, blocks, tr)` efectúa un procedimiento similar a ANOVA usando medias truncadas de las diferencias entre mediciones, mientras que la función `rmmcp(y, groups, blocks, tr, alpha)` implementa el procedimiento post-hoc para dicha prueba. Por otra parte, `rmanovab(y, groups, blocks, tr, nboot)` realiza la misma tarea que `rmanova()`, incorporando bootstrapping. En este caso, el procedimiento post-hoc está dado por la función `pairdepb(y, groups, blocks, tr, nboot)`. Los argumentos para esta familia de funciones son:

- `formula`: de la forma `<variable_dependiente> ~ <variable_independiente>`.
- `y`: vector con la variable dependiente.

- **groups**: vector que indica las medidas repetidas.
- **blocks**: vector que identifica los casos o bloques.
- **tr**: parámetro γ de la poda.
- **alpha**: nivel de significación.
- **nboot**: cantidad de repeticiones bootstrapping.

Consideremos, una vez más, la comparación del desempeño de los tres algoritmos implementados para el problema de ruteamiento de vehículos eléctricos con estaciones de carga intermedia (A , B y C), pero esta vez utilizando las mismas 25 instancias de igual tamaño y complejidad construidas aleatoriamente. El script 12.22 (líneas 7–14) muestra los tiempos de ejecución, en milisegundos, registrados para cada algoritmo al resolver cada presenta de estas instancias de prueba.

Script 12.22: datos y graficación de las diferencias de pares de observaciones del ejemplo.

```

1 library(dplyr)
2 library(ggpubr)
3 library(tidyr)
4 library(WRS2)
5
6 # Construir las estructuras con los datos
7 A <- c(32.0, 32.0, 32.0, 32.0, 32.1, 32.1, 32.2, 32.3, 32.3, 32.5, 32.7, 32.7,
8       32.7, 33.1, 33.4, 33.9, 34.1, 34.2, 34.5, 36.0, 36.6, 36.7, 37.2, 38.0)
9
10 B <- c(33.0, 33.0, 33.0, 33.0, 33.0, 33.0, 33.3, 33.3, 33.3, 33.3, 33.5, 33.6, 33.7,
11       33.9, 33.9, 34.2, 34.2, 34.3, 34.3, 34.4, 34.5, 34.6, 36.4, 38.9, 40.2)
12
13 C <- c(32.0, 32.2, 32.5, 32.6, 32.7, 32.7, 32.7, 33.0, 33.2, 33.4, 33.6, 33.6, 33.9,
14       34.1, 34.2, 34.4, 34.4, 34.5, 34.6, 34.7, 36.3, 36.6, 36.7, 38.9, 39.2)
15
16 datos anchos <- data.frame(Instancia = factor(1:length(A)), A, B, C)
17 dif anchos <- data.frame(A_B = A - B, A_C = A - C, B_C = B - C)
18
19 # Llevar las matrices de datos a formato largo
20 datos <- datos anchos |>
21   pivot_longer(c("A", "B", "C"), names_to = "Algoritmo", values_to = "Tiempo") |>
22   mutate(Algoritmo = factor(Algoritmo))
23 dif <- dif anchos |>
24   pivot_longer(everything(), names_to = "Algoritmos", values_to = "Diferencia") |>
25   mutate(Algoritmos = factor(Algoritmos))
26
27 # Obtener gráficos Q-Q de las diferencias
28 qq <- ggqqplot(dif, x = "Diferencia", facet.by = "Algoritmos",
29               color = "Algoritmos", palette = c("steelblue", "steelblue1", "steelblue4"),
30               xlab = "Cuantil teórico", ylab = "Diferencias en tiempos\nde ejecución [ms]")
31 qq <- qq + theme(legend.position = "none")
32 print(qq)

```

Considerando que esta comparación podría llevarse a cabo aplicando un procedimiento ANOVA para medidas repetidas, el script obtiene las diferencias pareadas entre cada par de algoritmos (línea 18 en formato ancho y líneas 25–27, en formato largo) y construye gráficos Q-Q para estas diferencias (líneas 30–34), en los que se observa la presencia de valores atípicos y desviaciones del comportamiento esperado (figura 12.25) si las distribuciones de origen de estas diferencias fueran normales. Luego, parece más aconsejable utilizar las alternativas robustas para analizar estos datos como alternativa al procedimiento ANOVA para muestras correlacionadas.

Como hemos discutido en los ejemplos anteriores, los tiempos de ejecución cumplen con tener escala de intervalos iguales y el experimento asegura una selección aleatoria de los casos de prueba (instancias), que no afectan en el tiempo requerido por otras instancias, garantizando así su mutua independencia.

Así, se cumplen las condiciones para aplicar un análisis robusto de una vía para muestras correlacionadas para contrastar, en el ejemplo, las siguientes hipótesis:

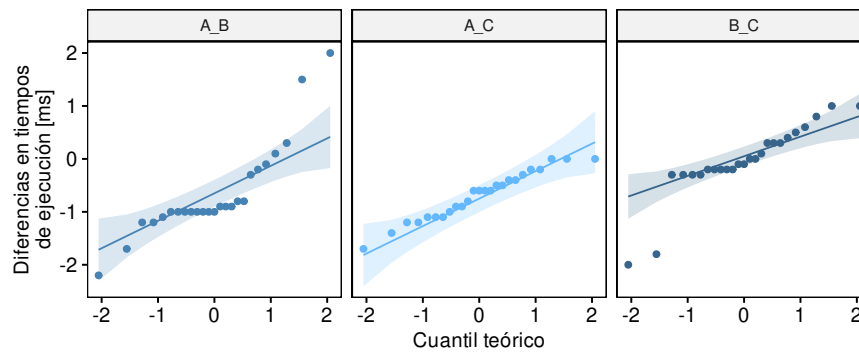


Figura 12.25: gráficos Q-Q de las diferencias de las observaciones apareadas del ejemplo.

H_0 : en promedio, no hay diferencias en el tiempo de ejecución necesitado por cada algoritmo en resolver las mismas instancias. Si $\mu_{(A-B)}$, $\mu_{(A-C)}$ y $\mu_{(B-C)}$ denotan las medias de las diferencias en tiempos de ejecución necesitado por cada par de algoritmos, entonces la hipótesis nula puede escribirse como: $\mu_{(A-B)} = \mu_{(A-C)} = \mu_{(B-C)} = 0$.

H_A : la media de las diferencias en el tiempo de ejecución necesitado para resolver las mismas instancias es diferente para al menos un par de algoritmos. Matemáticamente: $\exists X, Y \in \{A, B, C\} \mid \mu_{(X-Y)} \neq 0$.

El script 12.23 ilustra el funcionamiento de las funciones `rmanova()` y `rmmcp()` del paquete `WRS2` que aplican las pruebas robustas ómnibus y post-hoc, respectivamente, para el análisis de las medidas repetidas del ejemplo con un nivel de significación $\alpha = 0,05$ y de recorte $\gamma = 0,2$. Como para las muestras independientes, es importante tener en cuenta que el procedimiento post-hoc utiliza el método de Benjamini y Hochberg para el ajuste de los valores p de las comparaciones post-hoc entre todos los pares de mediciones (aunque esto no es reportado por la función `rmmcp()`). Otro aspecto a notar es que a esta función también entregamos el nivel de significación establecido (`alpha = alfa`) para que pueda estimar los correspondientes intervalos de confianza.

Script 12.23: (continuación del script 12.22) análisis robusto de una vía para muestras correlacionadas.

```

34 # Fijar nivel de significación y nivel de poda
35 alfa <- 0.05
36 gamma <- 0.2
37
38 # Comparar los algoritmos usando medias truncadas de las diferencias
39 mr_rob <- rmanova(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
40                 blocks = datos[["Instancia"]], tr = gamma)
41
42 cat("Análisis de una vía para medidas repetidas (asintótico)\n")
43 cat("-----\n")
44 print(mr_rob)
45
46 if(mr_rob[["p.value"]] < alfa) {
47   mr_rob_ph <- rmmcp(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
48                     blocks = datos[["Instancia"]], tr = gamma, alpha = alfa)
49
50   cat("Análisis post-hoc para medidas repetidas (asintótico)\n")
51   cat("-----\n")
52   print(mr_rob_ph)
53 }

```

La figura 12.26 contiene el resultado de ejecutar el script 12.23, que indica que la prueba robusta ómnibus es significativa para el ejemplo ($F(1,50; 20,96) = 24,171$; $p < 0,001$; $\gamma = 0,2$) al nivel establecido, por lo que debemos rechazar la hipótesis nula. Concluimos, entonces, con 95 % confianza, que existe una diferencia estadísticamente significativa entre los tiempos promedio de ejecución de los algoritmos cuando no se consideran casos extremos.

```

Análisis de una vía para medidas repetidas (asintótico)
-----
Call:
rmanova(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
        blocks = datos[["Instancia"]], tr = gamma)

Test statistic: F = 24.1706
Degrees of freedom 1: 1.5
Degrees of freedom 2: 20.96
p-value: 1e-05

Análisis post-hoc para medidas repetidas (asintótico)
-----
Call:
rmmcp(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]], blocks = datos[["Instancia"]],
      tr = gamma, alpha = alfa)

      psihat ci.lower ci.upper p.value p.crit  sig
A vs. B -0.85333 -1.16837 -0.53830 0.00000 0.0169 TRUE
A vs. C -0.68667 -0.98245 -0.39089 0.00002 0.0250 TRUE
B vs. C -0.00667 -0.26776  0.25443 0.94566 0.0500 FALSE

```

Figura 12.26: resultado del análisis robusto de una vía para las medidas repetidas del ejemplo.

Al efectuar el procedimiento post-hoc podando los casos extremos ($\gamma = 0,2$), podemos concluir que el algoritmo *A* es significativamente más eficiente que el algoritmo *B* ($\bar{x}_{(A-B)}^t = -0,853$ [ms]; 95 % CI: $[-1,168; -0,538]$ [ms]; $p < 0,001$) y también que el algoritmo *C* ($\bar{x}_{(A-C)}^t = -0,687$ [ms]; 95 % CI: $[-0,982; -0,391]$ [ms]; $p < 0,001$). Notemos que la función `rmmcp()` reporta valores *p* sin ajustar en la columna `p.value`, que compara contra un valor crítico, columna `p.crit`, que depende del orden de evaluación de cada prueba post-hoc. Así, si el valor *p* estimado es menor que el correspondiente valor *p* crítico, la prueba es considerada significativa.

Una forma de comprobar los resultados anteriores es usar las funciones `rmanovab()` y `pairdepb()` que aplican bootstrapping para el análisis robusto ómnibus y post-hoc con las medidas repetidas del ejemplo, como muestra el script 12.24. Debemos comentar que nuevamente no indicamos un método de ajuste para pruebas múltiples ni el nivel de significación para el cálculo de intervalos de confianza en la llamada al procedimiento post-hoc, puesto que la función `pairdepb()` utiliza un método integrado en el algoritmo de bootstrapping que considera el número de iteraciones simuladas para controlar la tasa de error por familia y un nivel de significación constante $\alpha = 0,05$.

También vale la pena mencionar que la función `rmanovab()`, a diferencia de las pruebas robustas vistas anteriormente, no estima un valor *p* ajustado, sino que un **valor crítico** para el estadístico de prueba *F*. De esta forma, si la prueba resulta o no significativa se determina comparando el valor obtenido para el estadístico con este valor crítico.

Script 12.24: (continuación del script 12.23) análisis robusto de una vía para muestras correlacionadas usando bootstrapping.

```

55 # Comparar los algoritmos usando diferencias truncadas y bootstrapping
56 nboot <- 999
57 set.seed(666)
58 mr_bt <- rmanovab(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
59                 blocks = datos[["Instancia"]], tr = gamma, nboot = nboot)
60
61 cat("Análisis de una vía para medidas repetidas (bootstrapped)\n")
62 cat("-----\n")
63 print(mr_bt)
64
65 if(mr_bt[["test"]] > mr_bt[["crit"]]) {
66   set.seed(666)
67   mr_bt_ph <- pairdepb(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
68                       blocks = datos[["Instancia"]], tr = gamma, nboot = nboot)
69
70   cat("Análisis post-hoc para medidas repetidas (bootstrapped)\n")
71   cat("-----\n")
72   print(mr_bt_ph)
73 }

```

```

Análisis de una vía para medidas repetidas (bootstrapped)
-----
Call:
rmanovab(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
  blocks = datos[["Instancia"]], tr = gamma, nboot = nboot)

Test statistic: 24.1706
Critical value: 4.4257
Significant: TRUE

Análisis post-hoc para medidas repetidas (bootstrapped)
-----
Call:
pairdepb(y = datos[["Tiempo"]], groups = datos[["Algoritmo"]],
  blocks = datos[["Instancia"]], tr = gamma, nboot = nboot)

      psihat ci.lower ci.upper      test      crit      sig
A vs. B -0.76000 -1.58621  0.06621 -4.59149  4.99147 FALSE
A vs. C -0.81333 -1.42198 -0.20469 -6.67012  4.99147  TRUE
B vs. C -0.05333 -0.52991  0.42325 -0.55859  4.99147 FALSE

```

Figura 12.27: resultado de procedimientos robustos para el análisis de una vía para medidas repetidas.

La figura 12.27 presenta la salida que genera el script 12.24, donde es posible observar que el valor crítico estimado con bootstrapping $F_{\text{crit}} = 4,4257$ es mucho menor que el estadístico de prueba observado $F = 24,1706$; confirmando la existencia de diferencias estadísticamente significativas en los tiempos de ejecución requeridos por los algoritmos cuando no se consideran casos extremos.

Análogamente, la función `pairdepb()` estima **un valor crítico** t_{crit} (columna `crit`) que utiliza para determinar la significación de cada prueba post-hoc comparando el estadístico obtenido en cada caso (columna `test`). Este procedimiento post-hoc con bootstrapping entrega una visión distinta a la obtenida con el equivalente asintótico. Vemos que se establece un valor crítico $t_{\text{crit}} = 4,99147$ considerando las 999 repeticiones bootstrap, el que solo es superado por la media recortada de las diferencias entre los algoritmos A y C : $\bar{x}_{(A-C)}^t = -0,813$ [ms]; 95 % CI*: $[-1,422; -0,205]$ [ms]; $t^* = 6,670$; $\gamma = 0,2$).

Consideremos nuevamente el gráfico Q-Q de las diferencias mostrado en la figura 12.25. Vemos que particularmente para las diferencias entre los algoritmos A y B se observan ciertas desviaciones de normalidad y asimetría que probablemente no se resuelven podando el 20 % de los casos de cada extremo, lo que podría estar afectando la calidad de la prueba asintótica. Por esta razón, en este caso, y en general con métodos asintóticos, los resultados basados en simulaciones de bootstrapping son más confiables. En consecuencia, concluimos con 95 % confianza que la evidencia es suficiente para establecer solamente que el algoritmo A es más eficiente que el algoritmo C cuando no se consideran casos extremos.

Nota importante

Debemos saber que el paquete `WRS2` implementa varios otros estimadores robustos y pruebas de hipótesis basadas en estos estimadores. Entre los no mencionados en esta sección, hay medidas y procedimientos robustos para obtener intervalos de confianza a partir de una muestra, alternativas para ANOVA de más de una vía, para modelos mixtos y para analizar regresión. También hay una familia de funciones que permite comparar variables aleatorias discretas (proporciones).

No es posible discutir las bases teóricas de todos estos métodos en esta sección, pero los desarrollos matemáticos y teoremas asociados a sus distribuciones muestrales se presentan en detalle en Wilcox (2012). Este libro se ha convertido en un referente de la estadística robusta, y presenta muchos más conceptos y métodos que los implementados en `WRS2`.

R. R. Wilcox no es el único investigador en esta área, como puede verse en la página de R dedicada al tema (Maechler, 2014).

12.3 EJERCICIOS PROPUESTOS

12.3.1 Técnicas de remuestreo (sección 12.1)

- 12.1 Investiga sobre la función `benchmark()` del paquete `rbenchmark` y comprueba que los tiempos de ejecución de las funciones del paquete `boot` son menores que los necesitados por las funciones del paquete `rsample` con un ejemplo propio.
- 12.2 En la sección 12.1.3, para obtener un valor p se desplazaron los valores de las muestras para que cumplieran la hipótesis nula previo a la aplicación de bootstrapping. Comprueba que realizando el desplazamiento de la distribución bootstrap obtenida para obtener el intervalo de confianza (es decir después de aplicar bootstrapping) se obtienen los mismos resultados.
- 12.3 Usando la búsqueda iterativa presentada en la sección 12.1.5 (página 232), obtén el intervalo con 95 % confianza para la diferencia de varianzas estudiada en la sección 12.1.6.
- 12.4 Esta búsqueda iterativa puede adaptarse para obtener un valor p para un intervalo de confianza estimado con Bootstrapping. Adapta el método iterativo presentado en la sección 12.1.5 (página 232) para obtener un valor p para el intervalo de confianza de tipo BCa obtenido en la sección 12.1.4.

En las siguientes preguntas prácticas se pide: enunciar las hipótesis para responder la pregunta planteada, verificar las suposiciones de una prueba paramétrica pertinente para contrastarlas y pronunciarse sobre su idoneidad; luego, independiente del resultado anterior, aplicar la técnica de remuestreo solicitada para obtener un intervalo de confianza o un valor p que permita dar una conclusión contextualizada. No olvidar que en caso de requerirse un análisis post-hoc, es importante que este se realice con las mismas remuestras usadas para la prueba ómnibus.

- 12.5 El conjunto de datos `diet` del paquete `WRS2` contiene datos de la pérdida de peso conseguida por tres tipos de dietas. Usando bootstrapping, determina si la pérdida de peso conseguida por las mujeres con las dietas A y C es la misma.
- 12.6 El conjunto de datos `bush` del paquete `WRS2` contiene datos de un “reality” australiano en que los participantes debieron comer palotes, ojos de pescado, testículos de canguro y larvas de una polilla. Específicamente se registró el tiempo que tardaron ocho participantes del programa en tener arcadas después de consumir cada uno de los tipos de alimentos. Determina si el tiempo que se tarda una persona en tener arcadas al comer estas cosas es el mismo usando bootstrapping.

Considera el siguiente enunciado:

El conjunto de datos `essays` del paquete `WRS2` se compone de datos recolectados por un estudio de los efectos de dos formas de retroalimentación sobre la calidad de la escritura académica producida por estudiantes de inglés como lengua extranjera en educación superior. Tres grupos de estudiantes, dos de tratamiento (las dos formas de retroalimentación) y uno de control, se formaron de forma aleatoria. Cada estudiante escribió cuatro ensayos: uno antes del tratamiento, uno de práctica durante el tratamiento, uno terminado el tratamiento y el último un mes después del tratamiento. Obviamente, estudiantes del grupo de control realizaron las tareas de escritura pero no recibieron retroalimentación.

y responde las siguientes preguntas:

- 12.7 Determina si una de las formas de retroalimentación estudiadas (directa o indirecta) es mejor que la otra (considera el ensayo 3 realizado al finalizar la intervención para este análisis) utilizando remuestreo con permutaciones.
- 12.8 Determina, a través de remuestreo con bootstrapping, si las y los estudiantes del grupo de control pudieron mejorar la tasa de errores cometidos en el tercer ensayo respecto del segundo.

- 12.9 Determina, usando remuestreo con permutaciones, si las y los estudiantes que recibieron retroalimentación directa mantuvieron la misma tasa de errores en el tercer y cuarto ensayo.
- 12.10 Determina, utilizando remuestreo con permutaciones, si la tasa de errores cometidos en el tercer ensayo son las mismas para cada uno de los tres grupos de estudiantes.
- 12.11 Determina con la técnica de bootstrapping si las tasas de errores mejoran al aplicar la retroalimentación indirecta y, si esto es así, este efecto se mantiene un mes después de la intervención.
- 12.12 Determina usando bootstrapping un intervalo con 95 % confianza para la probabilidad que un/a estudiante del grupo de control obtenga una tasa de errores menor a 1,5.
- 12.13 Determina usando permutaciones si ambas formas de retroalimentación llevan a la misma probabilidad de obtener una tasa de errores menor a 1 en el cuarto ensayo.
- 12.14 Determina, a través de remuestreo con bootstrapping, si la diferencia en la tasa de errores cometidos en el primer y cuarto ensayo son las mismas para cada uno de los tres grupos de estudiantes.

Considera el siguiente enunciado:

El conjunto de datos `birthwt` del paquete `MASS` contiene datos de recién nacidas/os en un hospital de Springfield (EE.UU.) registrando su peso junto a datos demográficos (raza: `race`, edad: `age`, fumadora: `smoke`) y de salud (peso inicial: `lwt`, historia de hipertensión: `ht`, presencia de irritabilidad uterina: `ui`) de la madre, además de si el peso al nacimiento fue bajo o no: `low`.

y responde las siguientes preguntas. Considera un nivel de significación $\alpha = 0,05$ y estos trozos de código genéricos para realizar el muestreo cuando sea solicitado.

```
1 library(dplyr)
2 # Selección aleatoria de n observaciones desde la muestra combinada
3 set.seed(semilla)
4 dataset |> select(factor, variable_dependiente) |> slice_sample(n = n)
5
6 # Selección aleatoria de n observaciones desde cada grupo (nivel del factor)
7 set.seed(semilla)
8 dataset |> select(factor, variable_dependiente) |> group_by(factor) |> slice_sample(n = n)
```

- 12.15 Usando una muestra aleatoria de 60 observaciones (fijando el valor 17 como semilla de reproducibilidad), determina si la edad media de las madres fumadoras es diferente de las madres no fumadoras. Usa remuestreo con permutaciones para el análisis estadístico.
- 12.16 Usando una muestra aleatoria de 50 observaciones (fijando el valor 11 como semilla de reproducibilidad), determina si, en promedio, los pesos iniciales de las madres fumadores y no fumadoras son diferentes. Usa remuestreo con bootstrapping para el análisis estadístico.
- 12.17 Usando una muestra aleatoria de 25 observaciones de cada raza (fijando el valor 11 como semilla de reproducibilidad), determina si las edades media de las madres son equivalentes. Usa remuestreo con permutaciones para el análisis estadístico.
- 12.18 Usando una muestra aleatoria de 20 observaciones de cada raza (fijando el valor 11 como semilla de reproducibilidad), determina si el peso inicial medio de las madres es diferente en cada una. Usa remuestreo con bootstrapping para el análisis estadístico.
- 12.19 Usando una muestra aleatoria de 40 observaciones (fijando el valor 17 como semilla de reproducibilidad), determina si la edad media de las madres que dieron a luz bebés con peso normal es diferente de las madres que tuvieron bebés con bajo peso. Usa remuestreo con bootstrapping para el análisis estadístico.
- 12.20 Usando una muestra aleatoria de 35 observaciones (fijando el valor 13 como semilla de reproducibilidad), determina si, en promedio, los pesos iniciales de las madres que dieron a luz bebés con peso normal no es diferente de las madres que tuvieron bebés con bajo peso. Usa remuestreo con permutaciones para el análisis estadístico.

Para las siguientes preguntas, se sigue usando el conjunto de datos `essays` descrito anteriormente, pero ahora el patrón de muestreo, si es requerido, es el siguiente:


```

1 library(dplyr)
2 # Selección aleatoria de n observaciones
3 set.seed(semilla)
4 dataset |> filter(<condición>) |> select(variable_dependiente) |> slice_sample(n = n)

```

Considera un nivel de significación $\alpha = 0,05$ para responder:

- 12.21 Usando una muestra aleatoria de 60 observaciones (fijando el valor 13 como semilla de reproducibilidad), determina si la irritabilidad uterina se da igualmente entre las madres que dieron a luz bebés con peso normal. Usa remuestreo con permutaciones para el análisis estadístico.
- 12.22 Usando una muestra aleatoria de 40 observaciones (fijando el valor 11 como semilla de reproducibilidad), determina si la frecuencia de nacimientos con bajo peso se da igualmente entre mujeres que no tienen historia de hipertensión. Usa remuestreo con bootstrapping para el análisis estadístico.
- 12.23 Usando una muestra aleatoria de 40 observaciones (fijando el valor 13 como semilla de reproducibilidad), determina si la proporción de madres sin historia de hipertensión que fuman es distinta a las que no lo hacen. Usa remuestreo con permutaciones para el análisis estadístico.
- 12.24 Determina si la presencia de hipertensión es independiente de la raza de las madres estudiadas. Usa remuestreo con bootstrapping para el análisis estadístico.
- 12.25 Determina si las frecuencias de madres fumadoras y no fumadoras se da de igual manera entre las diferentes razas de las madres estudiadas. Usa remuestreo con permutaciones para el análisis estadístico.
- 12.26 Las autoridades del recinto aseguran que en el 69 % de los nacimientos ocurridos en el hospital se reciben bebés con peso normal. Determina si los datos confirman esta declaración para las diferentes razas de las madres estudiadas. Usa remuestreo con bootstrapping para el análisis estadístico.

12.3.2 Métodos robustos (sección 12.2)

- 12.27 El conjunto de datos `diet` del paquete `WRS2` contiene datos de la pérdida de peso conseguida por tres tipos de dietas. Determina si la pérdida de peso conseguida por las mujeres con las dietas A y C es la misma.

Considera el conjunto de datos `essays` (descrito arriba de la pregunta 12.7) y responde las siguientes preguntas:

- 12.14 Determina si una de las formas de retroalimentación estudiadas (directa o indirecta) es mejor que la otra (considera el ensayo 3 realizado al finalizar la intervención para este análisis).
- 12.15 Determina si las y los estudiantes del grupo de control pudieron mejorar la tasa de errores cometidos en el tercer ensayo respecto del segundo.
- 12.16 Determina si las y los estudiantes que recibieron retroalimentación directa mantuvieron la misma tasa de errores en el tercer y cuarto ensayo.
- 12.17 Determina si la tasa de errores cometidos en el tercer ensayo son las mismas para cada uno de los tres grupos de estudiantes.
- 12.18 Determina si las tasas de errores mejoran al aplicar la retroalimentación indirecta y, si es así, si esta se mantiene un mes después de la intervención.
- 12.19 Considera el conjunto de datos `bush` (descrito en la pregunta 12.6), y determina si el tiempo que se tarda una persona en tener arcadas al comer estas cosas es el mismo usando métodos robustos.

12.4 BIBLIOGRAFÍA DEL CAPÍTULO

- Amat Rodrigo, J. (2016). *Resampling: test de permutación, simulación de Monte Carlo y Bootstrapping*. Consultado el 31 de mayo de 2021, desde https://www.cienciadedatos.net/documentos/23_resampling_test_permutacion_simulacion_de_monte_carlo_bootstrapping
- Hesterberg, T., Monaghan, S., Moore, D. S., Clipson, A., & Epstein, R. (2003). *Bootstrap Methods and Permutation Tests*. Consultado el 3 de junio de 2021, desde <https://statweb.stanford.edu/~tibs/stat315a/Supplements/bootstrap.pdf>
- Maechler, M. (2014). *CRAN task view: Robust statistical methods*. <https://cran.r-project.org/web/views/Robust.html>
- Mair, P., & Wilcox, R. (2020). Robust statistical methods in R using the WRS2 package. *Behavior Research Methods*, 52(2), 464-488.
- Scott, D. W. (1979). On optimal and data-based histograms. *Biometrika*, 66(3), 605-610.
- Wilcox, R. R. (2012). *Introduction to robust estimation and hypothesis testing* (3.^a ed.). Academic Press.